

Woord vooraf

Inhoudsopgave

Woord vooraf	1
Lijst van figuren	5
Abstract	7
Abstract in English	9
Inleiding	11
Achtergrond en gerelateerd werk	15
2.1 Fortunately-unfortunately methode	15
2.1.1 Beschrijving van de methode	15
2.1.2 Bestaande tool-ondersteuning	16
2.2 Gerelateerd werk	16
Ontwerp en implementatie	19
3.1 Ontwerpdoelstellingen	19
3.2 Architectuur en technologie	19
3.3 Niet-geïmplementeerde alternatieven	20
3.4 Geïmplementeerde en afgewezen lay-outs	21
3.5 Gekozen fileview-visualisatie	22
3.6 Basisfunctionaliteit en uitbreidingen	23
3.7 Node-representatie in de code	24
3.8 Statusiconen en danger rating	25
3.9 Branch summary en filterlogica	27
3.10 Hover-functionaliteit voor parent-context	31
3.11 Ondersteuning en feedback in de UI	32
3.12 Toepassing van Nielsen's usability-heuristieken in de webtool	33
3.12.1 Zichtbaarheid van de systeemstatus	33
3.12.2 Overeenkomst tussen systeem en de echte wereld	34
3.12.3 Gebruikerscontrole en vrijheid	34
3.12.4 Consistentie en standaarden	34
3.12.5 Foutpreventie	35
3.12.6 Herkenning boven herinnering	35
3.12.7 Flexibiliteit en efficiëntie van gebruik	36

3.12.8 Esthetisch en minimalistisch ontwerp	36
3.12.10 Helpen bij herkennen, diagnosticeren en herstellen van fouten.....	36
3.12.11 Help en documentatie.....	37
3.13 Prototypebeperkingen	37
Gebruikersstudie.....	39
4.1 Doel van de gebruikersstudie	39
4.2 Verloop van de gebruikersstudie	39
4.3 Deelnemers	40
4.4 Softwareversies	40
4.5 Taken en procedure	42
4.6 Dataverzameling	43
4.7 User Experience Questionnaire.....	43
4.8 Open vragen en rangschikking.....	43
Resultaten	45
5.1 Kwantitatieve UX-resultaten.....	45
5.2 Kwalitatieve observaties	46
5.3 Waardering van de toegevoegde features	47
5.4 Ervaren gebruiksgemak en voorkeur	47
5.5 Verbeterpunten en suggesties.....	47
5.6 Synthese.....	48
5.7 Samenvatting van de belangrijkste resultaten	48
5.8 Beantwoording van de onderzoeksvragen.....	48
5.8.1 Onderzoeksvraag 1: Minimale basisfunctionaliteit	48
5.8.2 Onderzoeksvraag 2: Meest nuttige extra functies	49
5.8.3 Onderzoeksvraag 3: Vergelijking met een basisversie.....	49
Discussie	51
6.1 Relatie met literatuur en theorie.....	51
6.2 Beperkingen van het onderzoek	52
6.3 Implicaties voor theorie en praktijk.....	53
6.4 Aanbevelingen voor toekomstig onderzoek.....	54
Conclusie	55
Referentielijst.....	57

Lijst van figuren

Fig. 1: Voorbeeld van een treemap	20
Fig. 2: Voorbeeld van een circulaire treemap	21
Fig. 3: Voorbeeld van een horizontale tree structuur.....	22
Fig. 4: Voorbeeld van een verticale tree structuur.....	22
Fig. 5: De huidige implementatie van de fileview geïnspireerde tree structuur	23
Fig. 6: De view van een fortunately node bij hoveren over de node	24
Figuur 7: De view van een unfortunately node bij hoveren over de node	25
Fig. 8: a) De iconbutton opties bij een fortunately node met kinderen, b) De iconbutton opties bij een fortunately node zonder kinderen, c) De iconbutton opties bij een unfortunately node zonder kinderen, d) De iconbutton optie bij een unfortunately node met kinderen.....	26
Fig. 9: De geselecteerde danger rating button met de opties impact en likelihood om de danger rating automatisch te berekenen	27
Fig. 10: De uitgeklapte branch summary sidebar bevat het pad van de root node naar de geselecteerde node.....	28
Fig. 11: De ingeklapte filter sidebar geeft het aantal nodes per status weer	29
Fig. 12: De uitgeklapte filter sidebar met de Assumption Under Review funtie toont alle nodes die deze status bevatten.....	29
Fig. 13: De uitgeklapte filter-sidebar met de Weakness Under Review-functie toont alle nodes met deze status, gesorteerd op danger rating	30
Fig. 14: De parent hover tooltip bij een node in de tree	31
Fig. 15: De parent hover tooltip bij een node in de filter sidebar	31
Fig. 16: De hover tooltip bij hoveren over de ingeklapte filter sidebar button	32
Fig. 17: De hover tooltip bij het hoveren over de iconbutton in de tree lay-out.....	32
Fig. 18: De hover tooltip bij het hoveren over de danger rating button in de tree lay-out.	33
Fig. 19: De hover tooltip bij het hoveren over de branch summary button in de tree lay-out.....	33
Fig. 20: De visualisatie van de tree structuur en de features in de basisversie van de software	41
Fig. 21: De fortunately-node weergaven in de basisversie van de software	41
Fig. 22: De unfortunately-node weergaven in de basisversie van de software	41
Fig. 23: Weergave van de UEQ resultaten in een grafiek met de gemiddelde waarden en hun t-test waarden	46

Abstract

Threat modeling is een systematische methode binnen softwareontwikkeling om potentiële beveiligingsrisico's vroegtijdig te identificeren en te mitigeren. Ondanks het belang ervan ervaren veel ontwikkelaars het proces als complex en tijdsintensief, wat vaak leidt tot onvolledige risicoanalyses. Dit onderzoek ontwikkelt en evalueert een gebruiksvriendelijke tool, gebaseerd op de Fortunately-Unfortunately-methode, om het threat modeling-proces toegankelijker en effectiever te maken.

Een literatuurstudie brengt bestaande threat modeling-raamwerken, mindmaptools en visualisatietechnieken in kaart. Op basis daarvan wordt een eisenlijst opgesteld met zowel generieke functies als mogelijkheden specifiek voor Fortunately-Unfortunately-treemodellering. Vervolgens wordt een webgebaseerd prototype ontwikkeld in React en D3, uitgebreid met zoek- en filterfuncties, visuele coderingen en ondersteunende interacties. De tool wordt geëvalueerd via een gebruikersonderzoek met negen deelnemers, die een scenario doorlopen, taken uitvoeren en hun ervaring rapporteren via een open vragenlijst en de User Experience Questionnaire (UEQ).

De resultaten tonen een duidelijke verbetering in gebruiksvriendelijkheid ten opzichte van een basis mindmapsoftware, met positieve UEQ-scores op alle dimensies (aantrekkelijkheid +1.04, efficiëntie +1.33, stimulatie +0.69) tegenover negatieve scores voor de basisversie. De tool verhoogt zo de toegankelijkheid van threat modeling voor niet-experts en biedt intuïtieve ondersteuning binnen het beveiligingsproces.

Abstract in English

Threat modeling is a systematic method within software development for identifying and mitigating potential security risks at an early stage. Despite its importance, many developers experience the process as complex and time-consuming, often resulting in incomplete risk analyses. This study develops and evaluates a user-friendly tool based on the Fortunately–Unfortunately method to make the threat modeling process more accessible and effective.

A literature review maps existing threat modeling frameworks, mind mapping tools, and visualization techniques. Based on these findings, a requirements list is defined, including both generic functionalities and features specific to Fortunately–Unfortunately tree modeling. A web-based prototype was then developed using React and D3, extending the tree structure with search and filter functions, visual encodings, and supportive interactions. The tool was evaluated through a user study with nine participants, who completed a scenario, performed predefined tasks, and reported their experiences using an open questionnaire and the User Experience Questionnaire (UEQ).

Results show a clear improvement in usability compared to a basic mind mapping software, with positive UEQ scores across all dimensions (attractiveness +1.04, efficiency +1.33, stimulation +0.69) versus negative scores for the basic version. The tool thus enhances the accessibility of threat modeling for non-experts and provides intuitive support within the security analysis process.

Hoofdstuk 1

Inleiding

Threat modeling is een systematische techniek om mogelijke dreigingen tegen een softwaresysteem te identificeren, te analyseren en te prioriteren nog vóór implementatie of tijdens evolutief onderhoud. In essentie dwingt threat modeling ontwikkelaars en architecten om expliciet na te denken over “wat kan er misgaan?”, “wat is de impact?” en “welke maatregelen zijn nodig?”, en vormt het daarmee een kernonderdeel van secure software engineering. Het proces omvat typisch het in kaart brengen van systeemcomponenten en dataflows, het identificeren van potentiële aanvallers en aanvalspaden, en het koppelen van passende mitigaties aan de belangrijkste risico's.

Binnen de softwareontwikkelingscyclus kan threat modeling op verschillende momenten worden ingezet, bijvoorbeeld tijdens requirementsanalyse, architectuurontwerp of bij grote refactorings. In een ideale DevSecOps-context wordt threat modeling iteratief uitgevoerd, parallel aan sprints, zodat nieuwe features meteen op beveiligingsimpact worden beoordeeld. In de praktijk blijkt dit echter moeilijk vol te houden: tijdsdruk, beperkte securityexpertise en een gebrek aan passende tooling zorgen ervoor dat threat modeling vaak wordt uitgesteld, oppervlakkig uitgevoerd of volledig overgeslagen.

Door de jaren heen zijn diverse methoden ontwikkeld om threat modeling te structureren. Een bekende categorie zijn taxonomie-gebaseerde benaderingen, zoals STRIDE, die dreigingen klasseren op basis van hun effect (bijvoorbeeld spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege). Andere benaderingen maken gebruik van attack trees of misuse cases, waarin ongewenst gedrag hiërarchisch wordt opgesplitst in deelstappen en varianten. In meer industriële context worden deze methoden vaak ondersteund door generieke diagramtools of mindmapsoftware, die wel flexibiliteit bieden maar weinig domeinspecifieke ondersteuning voor prioritering en statusopvolging.

Voor ontwikkelaars zonder diepgaande securityachtergrond vorm deze klassieke methoden een aanzienlijke drempel: ze vergen vertrouwdheid met beveiligingsconcepten, formele notaties en lijsten van standaarddreigingen, terwijl de beschikbare tools zelden gericht zijn op gebruiksgemak of leerbaarheid. Hierdoor dreigt threat modeling een specialistische activiteit te blijven, in plaats van een breed gedragen ontwerpraktijk binnen ontwikkelteams. Deze kloof tussen theoretisch

aanbevolen werkwijzen en feitelijke toepassing in de praktijk vormt een belangrijk motief voor onderzoek naar meer toegankelijke, visueel ondersteunde en gebruiksvriendelijke threat-modelingbenaderingen.

Dit onderzoek speelt in op deze drempels door een gebruiksvriendelijke webtool te ontwerpen en evalueren, gebaseerd op de Fortunately–Unfortunately-methode, waarin dreigingen en tegenmaatregelen in een intuïtieve boomstructuur worden voorgesteld. De tool integreert specifieke UX-features zoals kleurcodering, statusiconen, danger rating, branch summary en filtering om zowel navigatie als prioritering te ondersteunen en wordt geëvalueerd in een gebruikersonderzoek waarin de tool wordt vergeleken met een meer generieke (basis) referentiesoftware. Het doel van de masterproef is na te gaan in welke mate deze combinatie van visuele representatie en UX-features in combinatie met de Fortunately–Unfortunately-methode, threat modeling toegankelijker, efficiënter en effectiever maakt voor niet-security-experts. Om dit doel te bereiken, beantwoordt dit onderzoek de volgende onderzoeksvragen:

1. Welke basisfunctionaliteit is minimaal noodzakelijk om threat modeling met Fortunately-Unfortunately trees te ondersteunen?
2. Welke extra functies (bv. filters, zoekfuncties, statusiconen, visuele structuren) worden door gebruikers als het meest nuttig ervaren?
3. Hoe verhoudt de voorgestelde tool zich qua gebruiksvriendelijkheid en effectiviteit ten opzichte van basisversie van de mindmap software?

Het belang van dit onderzoek ligt enerzijds in het verlagen van de instapdrempel voor threat modeling, zodat ook ontwikkelaars zonder uitgesproken securityprofiel beter onderbouwde risicoanalyses kunnen uitvoeren. Anderzijds draagt het werk bij aan HCI-onderzoek rond gebruiksvriendelijke security-tools door empirisch te onderzoeken welke interface-elementen (zoals kleuren, iconen, filters en tooltips) daadwerkelijk bijdragen aan taakprestatie, gebruikservaring en feature-voorkeur. Daarmee sluit de masterproef aan bij een bredere verschuiving van puur technische security-oplossingen naar mensgerichte, visueel ondersteunde hulpmiddelen.

De rest van deze masterproef is als volgt opgebouwd. Hoofdstuk 2 bespreekt de theoretische achtergrond en gerelateerd werk rond threat modeling, de Fortunately–Unfortunately-methode en usability- en UX-evaluatie in securitycontexten. Hoofdstuk 3 beschrijft het ontwerp en de implementatie van het React/D3-webprototype en licht het onderzoeksdesign van de gebruikerstudie toe, inclusief taken en analysemethoden. Hoofdstuk 4 werkt de opzet van de gebruikersstudie uit, inclusief onderzoeksopzet, deelnemers, taken en analysemethoden. Hoofdstuk 5 presenteert de empirische resultaten over taakprestaties, user experience en feature-ranking, gevolgd door een discussie in hoofdstuk 6 waarin deze bevindingen worden geïnterpreteerd, gekoppeld

aan de literatuur en aangevuld met beperkingen en implicaties voor praktijk en toekomstig onderzoek. Hoofdstuk 7 besluit met de belangrijkste conclusies en schetst mogelijke uitbreidingen, zoals AI-ondersteuning en collaboratieve functies voor breder gebruik in onderwijs en industrie.

Hoofdstuk 2

Achtergrond en gerelateerd werk

2.1 Fortunately-unfortunately methode

2.1.1 Beschrijving van de methode

De Fortunately–Unfortunately-methode is een gestructureerde threat modeling-techniek die scenarios verhalend verkent door systematisch af te wisselen tussen gunstige en ongunstige ontwikkelingen. Oorspronkelijk afkomstig uit improvisatietheater, werd de methode door Adam Shostack onder de aandacht gebracht voor threat modeling in zijn paper Fast, Cheap and Good (2021), als snelle manier om in een heen-en-weer-dialoog mogelijke risico's en hun gevolgen rond een systeem te verkennen [1].

De methode omvat de volgende stappen:

1. Start met een fortunately node dat het doel van de te modelleren software beschrijft;
2. Bouw hierop verder door na te denken over mogelijk situaties die voor een probleem kunnen zorgen (risico's). Dit vormt nu een unfortunately node;
3. Bedenk een oplossing (mitigation) voor dit risico. Dit is een fortunately node;
4. Bouw verder aan de tree tot er een model gemaakt is dat alle risico's en oplossingen bevat. Dit is nu het security model voor de te maken software.

Een voorbeeld van een branch kan zijn:

- Fortunately: "Uitrol wachtwoordmanager verbetert gebruiksgemak [Assumption Under Review]";
 - Unfortunately: "Werknemers weigeren wachtwoordmanagers" (danger rating: 6) [Low Importance];
 - Fortunately: "Training verlaagt drempel" [Verified Assumption].

2.1.2 Bestaande tool-ondersteuning

Er zijn geen bestaande tools gevonden die de fortunately-unfortunately-methode expliciet ondersteunen. Bestaande threat modeling tools (Microsoft TMT, OWASP Threat Dragon) focussen op STRIDE-data flow diagrams of attack trees, zonder afdwinging van fortunately-unfortunately afwisseling. Generieke mindmapsoftware kan gebruikt worden, maar vereist handmatige toepassing van de fortunately-unfortunately afwisseling, zonder domein specifieke ondersteuning voor statussen of danger ratings.[2][3]

Shostack benadrukt de eenvoud van deze methode als een snelle manier om met beperkte security-skills toch aan threat modeling te doen [1]. Deze thesis vult dit aan met een fortunately-unfortunately-specifieke webtool die:

- Structuur technisch afdwingt (child nodes wisselen automatisch tussen fortunately en unfortunately type)
- Statusiconen (Weakness, Verified Assumption) en danger rating biedt
- Filtering en branch summaries toevoegt voor schaalbare navigatie

De Fortunately–Unfortunately-methode biedt verschillende voordelen en kent enkele beperkingen. De narratieve flow verlaagt de instapdrempel aanzienlijk voor niet-security-experts, terwijl de iteratieve ketens bestaande trade-offs tussen risico's en mitigerende maatregelen expliciet zichtbaar maken. Tegelijk is de inherent lineaire structuur minder geschikt voor het modelleren van niet-hiërarchische of parallelle risico's die gelijktijdig optreden binnen een systeem. [4]

2.2 Gerelateerd werk

Binnen dit onderzoek is de keuze van een passende boomvisualisatie cruciaal, omdat de Fortunately–Unfortunately-methode expliciet steunt op hiërarchische redeneringsstructuren. In de visualisatieliteratuur bieden treemap-technieken een rijk kader om de ontwerpkeuzes voor deze boom te situeren. Scheibel et al. presenteren een uitgebreid overzicht van 81 treemap-layoutalgoritmes en introduceren een uniforme terminologie voor boomstructuren (nodes, ancestors, depth, levels) en voor treemap-specifieke eigenschappen zoals containment, packing versus splitting en space-filling layouts. Zij tonen dat de meeste layouts zijn ontworpen voor 2D-rectangulaire vormen en vooral optimaliseren voor efficiënte benutting van schermruimte en het encoderen van kwantitatieve attributen via vlakgrootte. Tegelijk

benadrukken ze dat geen enkel algoritme in alle scenario's optimaal presteert en dat de keuze voor een layout altijd taak- en use-case-afhankelijk is.[5]

Voor dit werk is vooral relevant dat Scheibel et al. treemaps formeel definiëren als containment-gebaseerde layouts: elk kindvlak ligt volledig binnen het vlak van zijn ouder en siblings overlappen elkaar niet. Dit sluit aan bij de boomstructuur van de Fortunately–Unfortunately-tool, waar elke stap in het scenario genest is onder de voorafgaande beslissing. In tegenstelling tot klassieke treemaps wordt de beschikbare ruimte hier echter niet gebruikt om numerieke waarden (zoals bestandsgrootte of frequentie) te encodieren, maar om een sequentiële, causale keten van “fortunately”- en “unfortunately”-gevolgen leesbaar te maken. De survey maakt duidelijk dat bestaande treemap-algoritmes vooral ruimtegebruik, aspect ratio, buurtbehoud en stabiliteit over tijd optimaliseren, maar nauwelijks inspelen op de behoefte om expliciete argumentatie- of beslisstructuren in een security-context te tonen. Dit onderstreept een leemte: er is veel kennis over ruimtelijke verdeling van boomknooppunten, maar weinig over domeinspecifieke boomvisualisaties voor security-ontwerpargumentatie zoals in deze thesis.[5]

Waar treemap-werk zich richt op algemene hiërarchische data, focust Sbarski et al. specifiek op argumentstructuren. Zij beschrijven een visualisatietool voor argument mapping die een bipartite hi-tree introduceert, waarin proposities en “compound nodes” (redenen) elkaar afwisselen en compound nodes meerdere premissen groeperen voor multi-premise redeneringen. De tool gebruikt tekstuele labels (“support”, “oppose”) en kleurcodering (groen voor ondersteuning, rood voor tegenargumenten) om de evidentie relatie tussen knooppunten expliciet te maken. De auteurs kiezen bewust voor een aangepaste layered-tree-layout boven alternatieven zoals h-v trees, cone trees en radial trees, omdat die laatste de logische structuur minder duidelijk tonen en minder geschikt zijn voor grote, complexe argumentmaps. Bovendien bekritiseren zij bestaande argument-mappingtools om hun beperkte automatische layout, instabiele hertekeningen en gebrek aan controle over de volgorde van kinderen, wat direct raakt aan de leesbaarheid van redeneringen.[6]

De hi-tree-benadering is inhoudelijk verwant aan dit onderzoek: beide benaderingen willen inferentiële relaties (support/oppose versus fortunately/unfortunately) zichtbaar maken voor gebruikers die complexe redeneringen moeten begrijpen. Toch is het toepassingsdomein verschillend. Sbarski et al. richten zich op algemene argumentatie en kritische denkvaardigheden, terwijl deze thesis een specifieke vorm van security-ontwerpargumentatie modelleert, namelijk beslissingen over risico's en mitigaties in het kader van threat modeling. Deze focus op security-specifieke beslissingen vertaalt zich in extra semantiek in de boom (danger rating, statusiconen) en in UX-keuzes die zijn afgestemd op ontwikkelaars zonder diepe securityachtergrond, in plaats van op general-purpose argumentanalyse.[6]

Ten slotte plaatst de systematische review van Devendra et al. de visualisatie van security-informatie in een breder perspectief. Zij onderscheiden vier hoofdcategorieën van software security visualization: graph-based (bijvoorbeeld netwerk- en afhankelijkheidsgrafen), notation-based (zoals UML en ER-diagrammen), matrix-based (rechten- en risicomatrices) en metaphoric visualisaties (bijvoorbeeld 3D-“cityscapes”). De meeste van deze technieken richten zich op operationele security-aspecten, zoals netwerktrafiek, log-analyse, user behavior en CI/CD-integratie, en zijn primair ontworpen voor security-analisten en andere technische experts. De auteurs benadrukken dat deze visualisaties vaak moeilijk te integreren zijn in agile pipelines en dat er een gebrek is aan gebruikersvriendelijke, taakgerichte visualisaties voor niet-experts.[7]

Het werk in deze thesis sluit conceptueel aan bij deze review doordat het eveneens security-relevante informatie visualiseert, maar verschilt wezenlijk in focus: het gaat niet om monitoring van logs, phishingcampagnes of runtime-gedrag, maar om de visualisatie van security-ontwerpargumentatie en beslissingen tijdens threat modeling. Waar Devendra et al. voornamelijk bestaande tools voor operationele analyse synthetiseren, verkent deze thesis een narratieve, boomgebaseerde representatie die de trade-offs tussen risico's (unfortunately) en mitigaties (fortunately) expliciet maakt en daarmee het ontwerpproces zelf ondersteunt. Daarmee positioneert dit werk zich op het snijvlak van hiërarchische visualisatie (treemap-design space), argumentvisualisatie (hi-trees) en security-visualisatie, maar met een nadrukkelijke verschuiving van operationele analyse naar begrijpelijke ontwerpargumentatie voor niet-security-experts.[6][7]

Hoofdstuk 3

Ontwerp en implementatie

In dit hoofdstuk wordt het ontwerp en de implementatie van de threat-modelingtool beschreven. Eerst wordt de gekozen architectuur en gebruikte technologieën toegelicht, om te verantwoorden waarom deze stack geschikt is voor interactieve boomvisualisaties en snelle iteratie. Vervolgens worden alternatieve visualisatielay-outs besproken en gemotiveerd, zodat duidelijk wordt waarom uiteindelijk voor de fileview-representatie is gekozen. Daarna volgt een gedetailleerde beschrijving van de belangrijkste interface-componenten en UX-features (kleurcodering, statusiconen, filtering, branch summaries), die samen de ondersteuning voor de Fortunately–Unfortunately-methode concretiseren. Tot slot worden prototypebeperkingen en designkeuzes samengevat als voorbereiding op de gebruikersstudie in het volgende hoofdstuk.

3.1 Ontwerpdoelstellingen

Het ontwerp van de webtool vertrekt vanuit de nood om threat modeling concreet uitvoerbaar te maken voor ontwikkelaars zonder uitgebreide securityachtergrond, met een sterke focus op leesbaarheid van de boom, directe visuele feedback en eenvoudige navigatie. De codebasis is daarom opgezet rond een beperkt aantal, duidelijk herkenbare visuele bouwblokken (nodes, iconen, panelen) die in de volledige interface consequent worden hergebruikt. De Fortunately–Unfortunately-structuur wordt afgedwongen op code-niveau door het node-type expliciet op te slaan en in de renderlogica te gebruiken om kleuren, iconen en interactie te bepalen.

3.2 Architectuur en technologie

De tool is geïmplementeerd als een single-page webapplicatie met React en next.js voor de componenthiërarchie en D3-functionaliteit voor de boomlay-out en interactie. De kern van de applicatie bestaat uit:

- een datastructuur voor nodes (name, children, danger rating, level, status, statusColor);

- een visualisatielaag met React-componenten voor nodes, iconen, zijpanelen en tooltips;
- hulpfuncties om paden naar de root te berekenen (voor branch summary) en gefilterde lijsten op basis van status en danger rating samen te stellen.

De data worden in een boomstructuur bijgehouden, terwijl D3 wordt gebruikt om posities te berekenen en transities (uitklappen, inklappen) te ondersteunen. React zorgt ervoor dat elke wijziging in de onderliggende data (level, status, statusColor) onmiddellijk leidt tot een hertekening van de corresponderende componenten.

3.3 Niet-geïmplementeerde alternatieven

Treemaps (Fig. 1) en circulaire treemaps (Fig. 2) werden theoretisch geëvalueerd aan de hand van bestaande voorbeelden en literatuur over hiërarchische visualisatie. Circulaire lay-outs, waarin nodes rond een centrale root worden gerangschikt met unfortunately-nodes prominent op basis van danger rating, bleken ongeschikt omdat sequentiële fortunately/unfortunately-relaties visueel fragmentarisch worden en parent-child-paden cognitief moeilijk traceerbaar blijven. Treemaps, met unfortunately-nodes als dominante blokken en fortunately-nodes als subregio's geschaald op danger rating, prioriteren risico-overzicht maar schenden de lineaire causaliteitslogica van de methode en verhogen de cognitieve belasting bij diepere bomen. Op basis van deze analyse werden deze technieken niet geïmplementeerd.

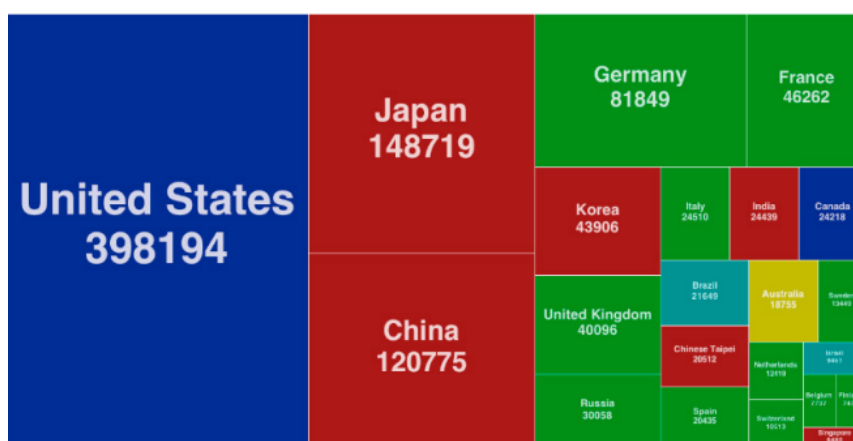


Fig. 1: Voorbeeld van een treemap

West, M.. Tree Maps <http://mjwest.org/vis/treemaps/> (last access 12/05/2017)

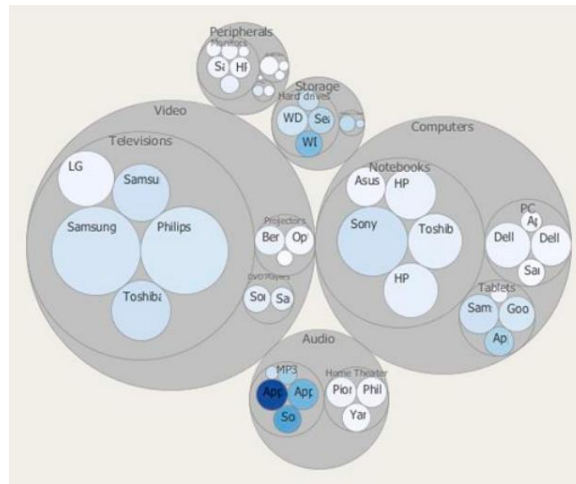


Fig. 2: Voorbeeld van een circulaire treemap

Fischer, F. Fuchs, J. and Mansmann, F., ClockMap: Enhancing Circular Treemaps with Temporal Glyphs for Time-Series Data. In Proceedings of the Eurographics Conference on Visualization (EuroVis 2012), 2012

3.4 Geïmplementeerde en afgewezen lay-outs

Horizontale tree lay-outs (Fig. 3) en verticale tree lay-outs (Fig. 4) werden concreet geïmplementeerd en intern geëvalueerd. De horizontale variant visualiseert fortunately/unfortunately-alternering van links naar rechts, terwijl de verticale variant een top-down hiërarchie met uitklapbare takken volgt. Beide implementaties gebruikten dezelfde node-componenten en UX-features, maar resulteerden in suboptimale prestaties: excessieve witruimteverbruik, verminderde leesbaarheid van lange sequenties en ontoereikende ondersteuning voor zowel globale oriëntatie als lokale detailanalyse. Deze lay-outs werden daarom verworpen ten gunste van een alternatieve fileview-representatie.

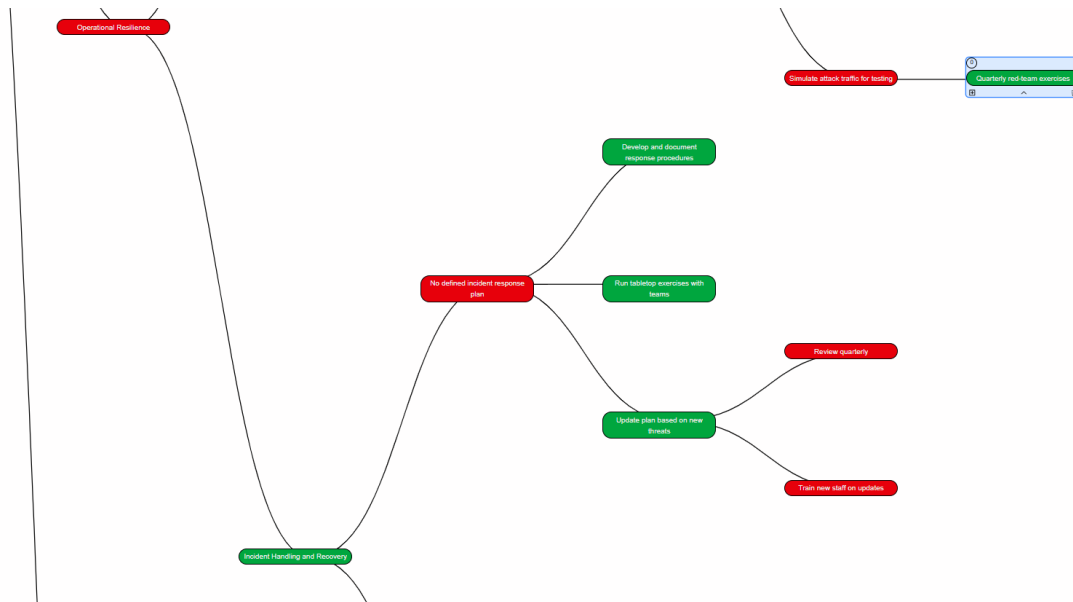


Fig. 3: Voorbeeld van een horizontale tree structuur

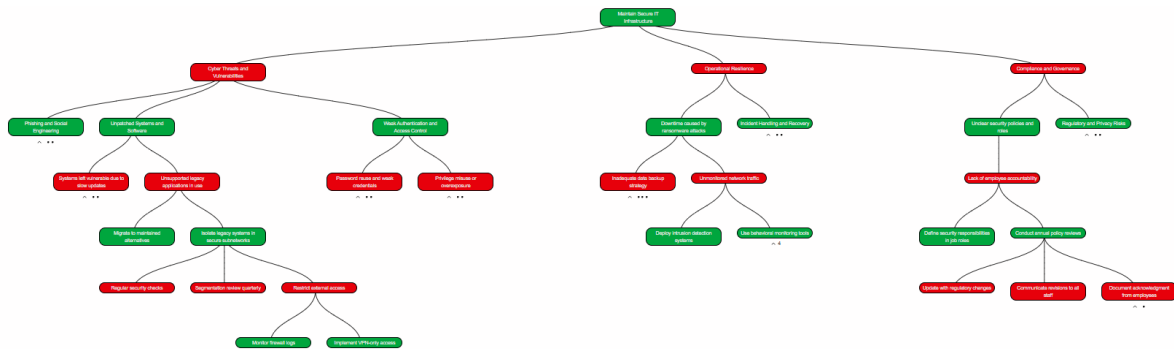


Fig. 4: Voorbeeld van een verticale tree structuur

3.5 Gekozen fileview-visualisatie

De definitieve visualisatie maakt gebruik van een fileview geïnspireerde lay-out (Fig. 5), waarin de boom als een hiërarchische bestandsstructuur wordt voorgesteld met consistente node-weergave en minimale witruimte. Deze keuze balanceert sequentiële leesbaarheid met schaalbaarheid en werd exclusief gebruikt tijdens de gebruiker studie.



Fig. 5: De huidige implementatie van de fileview geïnspireerde tree structuur

3.6 Basisfunctionaliteit en uitbreidingen

Dit hoofdstuk beschrijft het ontwerp van de threat-modelingtool als combinatie van een minimale kern en een reeks UX-gerichte uitbreidingen. De basisfunctionaliteit omvat alle elementen die nodig zijn om Fortunately–Unfortunately-trees überhaupt te kunnen modelleren: het aanmaken, hernoemen en verwijderen van nodes, het uit- en inklappen van takken, een consistente boom-layout (fileview-representatie) en een eenvoudige zoekfunctie over node-namen. Deze kern is onafhankelijk van de extra visualisatie- en analyselagen en komt overeen met wat ook in generieke mindmap- of diagramsoftware terug te vinden is.

Bovenop deze basislaag zijn vervolgens uitbreidingen ontworpen die specifiek inspelen op threat modeling met de Fortunately–Unfortunately-methode. Het gaat onder meer om kleurcodering voor fortunately/unfortunately-nodes, statusiconen voor weaknesses en aannames, danger ratings op basis van likelihood en impact, filtering en sorteren in een zijbalk, highlighten van geselecteerde nodes en branch summaries. In de rest van dit

hoofdstuk wordt eerst de implementatie van de basisfunctionaliteit toegelicht (architectuur, fileview-layout), waarna de verschillende uitbreidingen afzonderlijk worden besproken. Deze opdeling maakt het mogelijk om in de resultaten en discussie duidelijk te onderscheiden welke effecten toe te schrijven zijn aan de minimaal noodzakelijke functionaliteit en welke aan de ontworpen toevoegingen.

3.7 Node-representatie in de code

De node-representatie is ontworpen om de afwisseling tussen fortunately- en unfortunately-stappen visueel onmiddellijk duidelijk te maken en tegelijk de boom goed leesbaar te houden bij lange ketens. Elke node toont daarom steeds dezelfde basiselementen (tekst, rand, vorm), maar het type wordt onderscheidend weergegeven via kleur en label. De fortunately-nodes (Fig. 6) krijgen een groene achtergrond en unfortunately-nodes (Fig. 7) een rode achtergrond, telkens met afgeronde hoeken, zwarte rand en gecentreerde tekst. Deze consistente vormgeving ondersteunt snelle scanning van de structuur en maakt het eenvoudig om in één oogopslag te zien waar positieve en negatieve stappen elkaar afwisselen, wat cruciaal is voor het begrijpen van de logica van een Fortunately–Unfortunately-tree.

Alle nodes worden via één generieke component opgebouwd, zodat maximum breedte, tekstomloop, padding en borders overal identiek zijn. Dit voorkomt visuele ruis en zorgt ervoor dat gebruikers zich kunnen concentreren op de inhoud (de scenario-stappen) in plaats van op kleine stijlverschillen. Boven elke node wordt in kleinere typografie het corresponderende label “fortunately” of “unfortunately” toegevoegd. Deze redundante codering (kleur én tekst) verlaagt de cognitieve belasting, sluit aan bij Nielsen’s heuristieken rond consistentie en herkenning boven herinnering, en komt ook gebruikers met kleurperceptieproblemen tegemoet, omdat zij niet uitsluitend op kleur hoeven te vertrouwen.

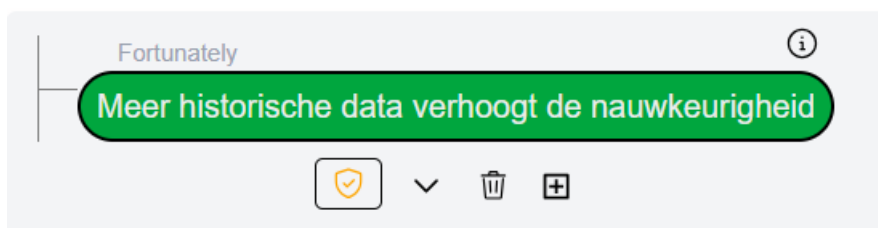
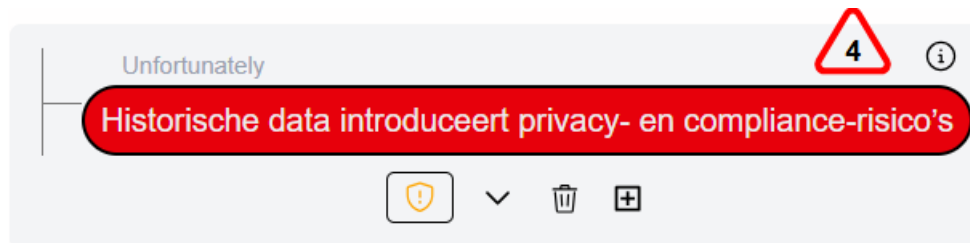


Fig. 6: De view van een fortunately node bij hoveren over de node



Figuur 7: De view van een unfortunately node bij hoveren over de node

3.8 Statusiconen en danger rating

Statusiconen en danger ratings zijn toegevoegd om in één oogopslag te tonen welke onderdelen van het model nog aandacht vragen en hoe kritisch ze zijn. In de basisversie moesten gebruikers dit zelf onthouden of in de node-tekst noteren, wat foutgevoelig is bij grotere trees. Daarom krijgt elke node nu een statusicoon dat tegelijk het type en de voortgang uitdrukt: een rood schild voor Weakness, een oranje schild voor Weakness Under Review, een oranje checkschild voor Assumption Under Review en een groen checkschild voor Verified Assumption. Daarnaast is er een blauw checkschild voor volledig afgewerkte fortunately-nodes of voor unfortunately-nodes met lage prioriteit, zodat gebruikers ook minder belangrijke of afgeronde paden expliciet kunnen markeren. De statusopties in de icon-knop zijn bovendien contextafhankelijk en sluiten aan bij de rol van de geselecteerde node in de boom. Een fortunately-node met kinderen kan zo enkel als Assumption Under Review of als een Completed node worden gemarkeerd (Fig. 8a), terwijl een fortunately-node zonder kinderen als Assumption Under Review of als Verified Assumption kan worden ingesteld (Fig. 8b). Een unfortunately-node zonder kinderen kan de status Weakness of Low Importance krijgen (Fig. 8c), terwijl een unfortunately-node met kinderen wordt behandeld als Weakness Under Review of als een Completed node (Fig. 8d). Daarnaast is gekozen voor een schild-metafoor omdat dit visueel sterk geassocieerd wordt met beveiliging en bescherming, waardoor gebruikers de link met security-statussen intuïtief leggen zonder extra uitleg.

Er is bewust gekozen voor compacte iconen in plaats van louter tekst labels, omdat symbolen en kleuren sneller visueel scanbaar zijn dan woorden en minder schermruimte innemen, wat vooral bij grote trees belangrijk is. Deze keuze sluit aan bij UX-principes rond zichtbaarheid van de systeemstatus en herkenning boven herinnering: gebruikers kunnen de betekenis van een status op basis van vorm en kleur herkennen zonder steeds volledige labels te moeten lezen. Deze visuele codering maakt het makkelijker om tijdens een sessie snel te zien waar nog beslissingen of acties nodig zijn en ondersteunt collaboratieve communicatie binnen het team. Een nadeel van deze

keuze is dat puur kleur gecodeerde iconen minder toegankelijk kunnen zijn voor kleurenblinde gebruikers. Dit wordt deels opgevangen door consistente vormverschillen (schild versus checkschild) en door de mogelijkheid om de betekenis van een icoon via een tooltip op te vragen, maar toekomstige iteraties kunnen nog explicietere textuele labels of alternatieve patronen toevoegen om de toegankelijkheid verder te verbeteren.

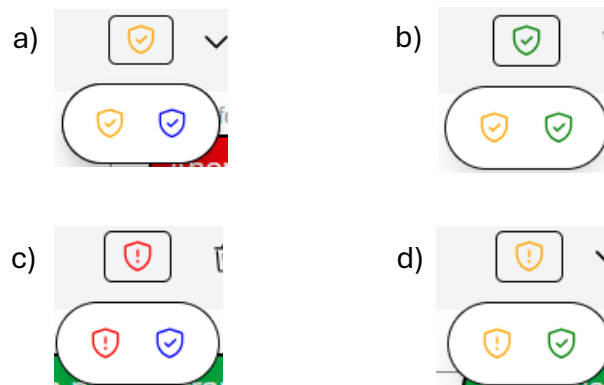


Fig. 8: a) De iconbutton opties bij een fortunately node met kinderen, b) De iconbutton opties bij een fortunately node zonder kinderen, c) De iconbutton opties bij een unfortunately node zonder kinderen, d) De iconbutton optie bij een unfortunately node met kinderen

De danger rating vult deze statussen aan met een kwantitatieve inschatting van risico. Bij het selecteren van de danger rating button worden de impact- en likelihood-knoppen zichtbaar (Fig. 9), waarmee de gebruiker elk een waarde van 1 tot 5 kan toekennen. Op basis van deze twee waarden wordt automatisch de danger rating berekend. Deze waarde wordt permanent weergegeven bij elke unfortunately-node en gebruikt om nodes in de filterweergave aflopend te sorteren, zodat de ernstigste weaknesses automatisch bovenaan verschijnen. De danger rating button wordt bovendien visueel gekoppeld aan een waarschuwingsdriehoek-icoon, een symbool dat in het dagelijks leven vaak gebruikt wordt om gevaar of verhoogde aandacht aan te duiden. Deze keuze versterkt de intuïtieve interpretatie van de score als risicosignaal en helpt gebruikers om kritieke weaknesses sneller te herkennen zonder bijkomende uitleg. Door de rating altijd zichtbaar te houden, in plaats van ze achter extra kliks te verbergen, wordt prioriteren sterk vereenvoudigd en hoeven gebruikers minder te onthouden. Tegelijk is gekozen om secundaire acties (zoals add, delete en collapse) pas bij hover te tonen, om visuele overbelasting te vermijden en de focus op de kerninformatie, status en gevaar, te leggen. De centrale berekening en hergebruik van dezelfde rating in zowel de boom als de filterlijsten zorgen voor consistentie in de interface en sluiten aan bij usability-principes rond zichtbaarheid van systeemstatus en herkenning boven herinnering.

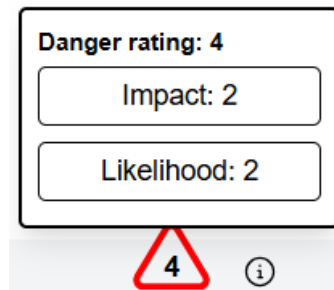


Fig. 9: De geselecteerde danger rating button met de opties impact en likelihood om de danger rating automatisch te berekenen

3.9 Branch summary en filterlogica

De branch-samenvatting en filterfunctionaliteit zijn toegevoegd om twee terugkerende problemen bij het werken met grotere Fortunately–Unfortunately-trees op te lossen: het verlies van overzicht over de volledige redeneringsketen en de moeilijkheid om snel naar de belangrijkste weaknesses en assumptions te navigeren. In de basisversie moeten gebruikers hiervoor handmatig door de boom scrollen en zelf bijhouden welke paden relevant waren. Dit kan tijdrovend en foutgevoelig zijn bij complexere modellen. De zijpanelen voor branch summary en filtering bieden daarom enerzijds globale context (waar bevind ik mij in de tree?) en anderzijds taakgerichte prioritering (welke nodes moet ik nu eerst bekijken?).

De branch summary reconstrueert het pad van de rootnode tot de geselecteerde node en toont dit als compacte lijst in een zijpaneel (Fig. 10). Hierdoor kan een gebruiker tijdens een sessie snel zien welke fortunately- en unfortunately-stappen aan een bepaalde beslissing voorafgaan, zonder telkens terug te moeten scrollen naar het begin van de tree. Dit ondersteunt typische taken zoals het voorbereiden van een samenvatting richting een teamlead of het controleren of een bepaalde mitigatie wel op de juiste plaats in de keten hangt. De samenvatting vult zo de fileview-representatie aan met een lineaire, leesbare weergave van de context, wat aansluit bij UX-principes rond oriëntatie en minimalisering van onnodige navigatiestappen. Doordat deze context in een aparte maar gekoppelde visualisatie wordt aangeboden, kunnen gebruikers de volledige redeneringsketen los van de detailweergave bestuderen, wat het begrijpen en bespreken van de logica binnen een specifieke branch verder vergemakkelijkt.

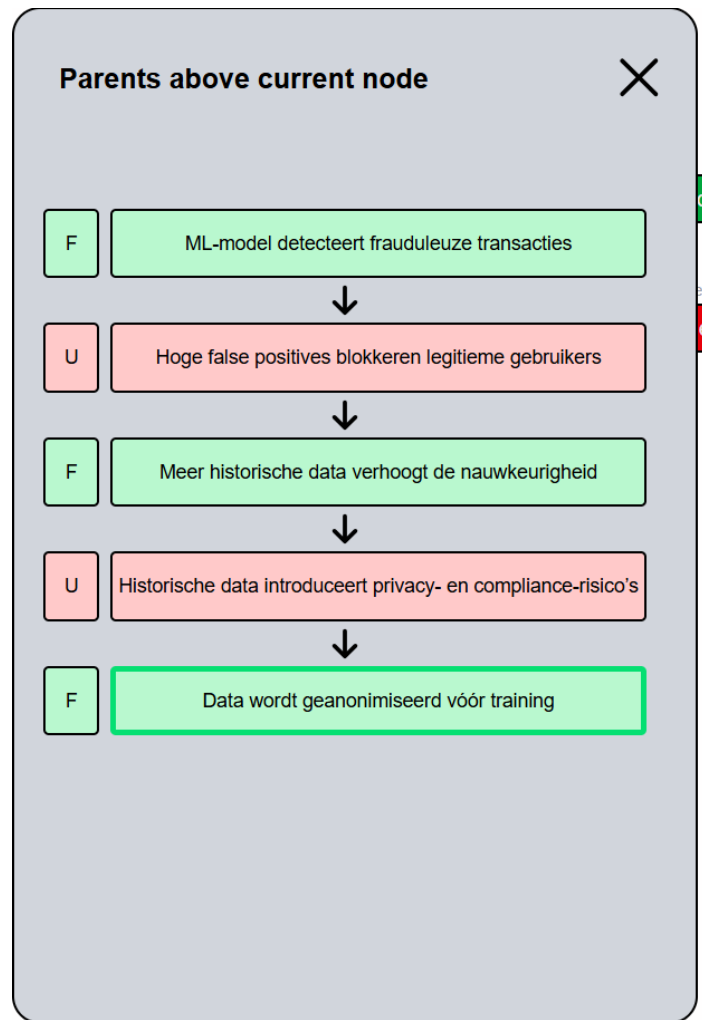


Fig. 10: De uitgeklapte branch summary sidebar bevat het pad van de root node naar de geselecteerde node

De filterlogica is ontworpen om gebruikers direct naar de belangrijkste risico's te leiden. In ingeklapte toestand toont de filterbalk bovendien per statuscategorie een compacte schild-knop met het aantal bijhorende nodes (Fig. 11), waardoor gebruikers in één oogopslag een overzicht hebben van de globale voortgang van het model zonder extra schermruimte in te nemen. In de uitgeklapte filter sidebar worden nodes gegroepeerd per statuscategorie (Weakness, Weakness Under Review, Assumption Under Review, Verified Assumption) (Fig. 12). Binnen de Weakness en Weaknesss under review worden de nodes op aflopende danger rating gesorteerd (Fig. 13). Hierdoor verschijnen de meest kritieke weaknesses automatisch bovenaan en kan een analist met één klik naar de relevante plaats in de boom springen, waarbij de gekozen node visueel wordt gehighlight. Dit ontwerp sluit aan bij de doelstelling om threat modeling efficiënter te maken: in plaats van handmatig te zoeken, kunnen gebruikers systematisch “top-down” door hun werkvoorraad van openstaande risico's en aannames gaan.

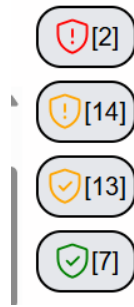


Fig. 11: De ingeklapte filter sidebar geeft het aantal nodes per status weer

Weaknesses

Weaknesses Under Review

Assumptions Under Review

Verified assumptions

ML-model detecteert frauduleuze transacties

Meer historische data verhoogt de nauwkeurigheid

Data wordt geanonimiseerd vóór training

Feature engineering verbetert prestaties op een veilige manier

Realtime waarschuwingen voorkomen financiële schade

Systeem optimaliseert door laag-risico transacties te batchen

Hardware wordt opgeschaald tijdens piekmomenten

Team voert uitlegbaarheidstools toe om modelbeslissingen te verduidelijken

Fig. 12: De uitgeklapte filter sidebar met de Assumption Under Review functie toont alle nodes die deze status bevatten

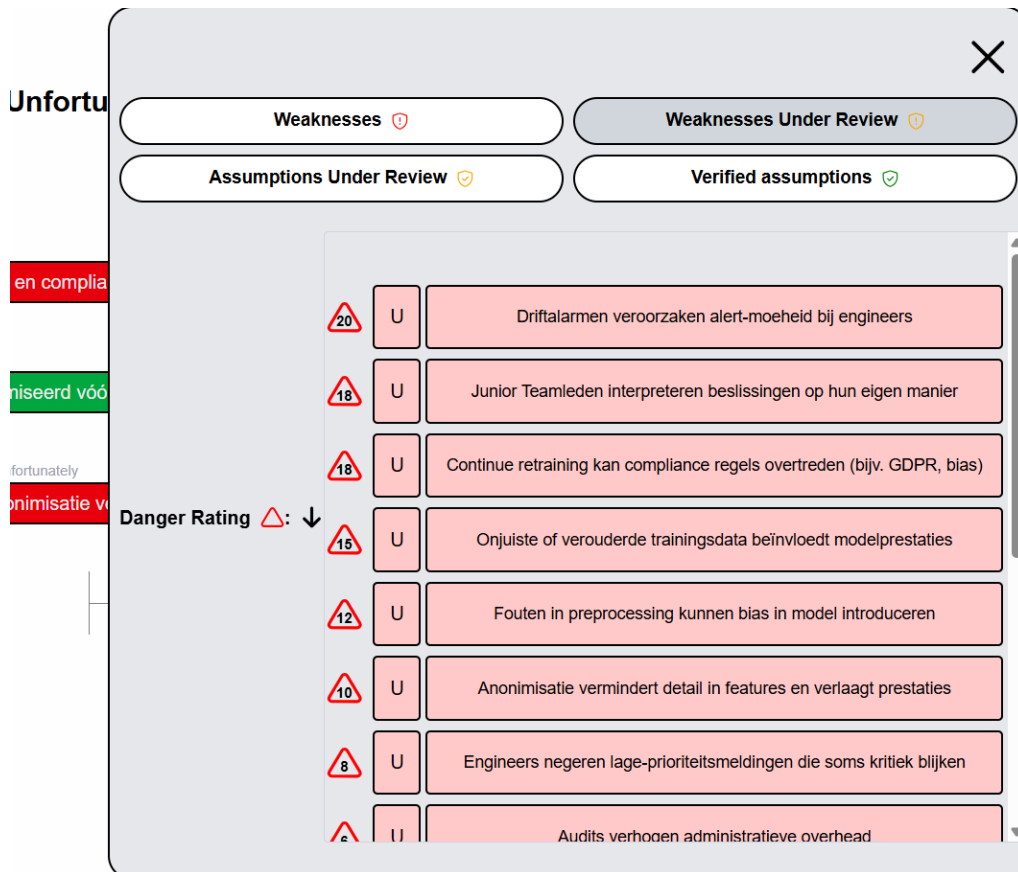


Fig. 13: De uitgeklapte filter-sidebar met de Weakness Under Review-functie toont alle nodes met deze status, gesorteerd op danger rating

Bij het ontwerp is ook overwogen om deze functies op een aparte pagina of in een popup-venster onder te brengen. Dit alternatief zou het zicht op de boom volledig of gedeeltelijk wegnemen en maakt het minder duidelijk naar welke exacte plaats in de structuur wordt genavigeerd wanneer een node geselecteerd wordt. Hierdoor neemt de ruimtelijke oriëntatie van de gebruiker af. Dergelijke oplossing zou bovendien meer cognitieve schakelmomenten vereisen tussen lijst en boom en minder direct aangeven welke items het meest urgent zijn. Door filtering en sortering te integreren in een zijpaneel dat direct naar nodes navigeert, wordt voldaan aan heuristieken rond zichtbaarheid van systeemstatus en herkenning boven herinnering. De tool toont expliciet welke weaknesses bestaan, welke nog “under review” zijn en welke veronderstellingen al geverifieerd zijn, zonder dat gebruikers dit zelf moeten onthouden. Een mogelijke beperking is dat het zijpaneel extra schermruimte inneemt en dat gebruikers de betekenis van de verschillende statussen moeten leren. Dit wordt deels opgevangen door consistente kleur- en iconencodering en door het fortunately/unfortunately-label op de nodes, maar verdere onboarding of een korte legend kan in toekomstig werk nog voorzien worden.

3.10 Hover-functionaliteit voor parent-context

In zowel de boomvisualisatie als de filter sidebar, wordt bij hover over een node de directe parent getoond (Fig. 14 en Fig. 15). Zo is de hiërarchische context expliciet zichtbaar tijdens de interactie. Dit ontwerp speelt in op het doel om gebruikers ook in grotere trees snel te laten oriënteren. In de boom kunnen zij zonder terug te scrollen zien in welke redeneringsstap een node ingebed is, terwijl de filter sidebar toelaat om gefilterde nodes (bijvoorbeeld alle Weaknesses) te beoordelen zonder eerst hun exacte positie in de structuur te moeten reconstrueren. Door deze context als korte parent-referentie te tonen, in plaats van bijvoorbeeld volledige breadcrumb-paden of extra tekstlabels, blijft de interface visueel licht. Op deze manier wordt de cognitieve belasting beperkt zonder cruciale relatie-informatie te verliezen. Een nadeel is dat enkel de directe parent zichtbaar is en niet de volledige keten. Dit kan in uitzonderlijk diepe trees extra navigatie en een expliciete handeling van de gebruiker vergen om via de branch-summary sidebar de volledige context op te vragen. Dit werd aanvaard als compromis om schermruimte en visuele ruis te beperken, in lijn met UX-principes rond minimalisme en gerichte informatievoorziening.

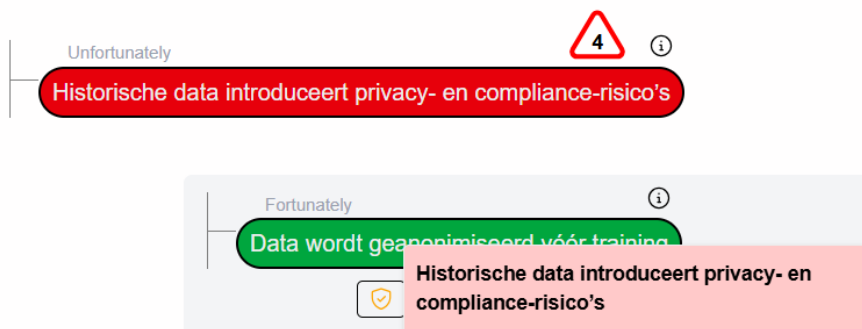


Fig. 14: De parent hover tooltip bij een node in de tree

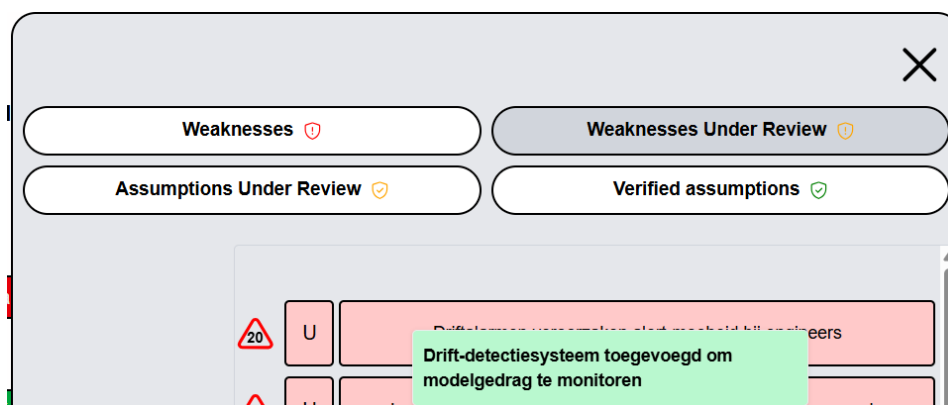


Fig. 15: De parent hover tooltip bij een node in de filter sidebar

3.11 Ondersteuning en feedback in de UI

De ondersteuning voor gebruikers is bewust uitgewerkt met hover-tooltips en consistente iconografie om de interface informatief te maken zonder extra schermruis toe te voegen. Korte, vaste teksten in de tooltips leggen bij hover de betekenis van statussen (Fig. 16 en Fig. 17) en symbolen uit (Fig. 18 en Fig. 19). Zo kunnen nieuwe gebruikers de codering stapsgewijs leren terwijl ervaren gebruikers niet worden vertraagd. Deze keuze gaat uit van het doel om “herkenning boven herinnering” te ondersteunen. De cruciale uitleg is altijd beschikbaar op de plek waar de gebruiker zich bevindt, zonder dat zij een aparte handleiding moeten raadplegen. Node-selectie, statuswijzigingen en filteracties geven directe visuele feedback via kleuraccenten, borders of highlight-effecten, waardoor gebruikers onmiddellijk zien wat er in de boom is veranderd. Hierdoor zijn er geen bijkomende modale meldingen nodig. Een beperking is dat gebruikers wel moeten weten dat zij kunnen hoveren om extra uitleg te krijgen. Dit werd aanvaard als compromis om de interface visueel zo eenvoudig mogelijk te houden. Er is bovendien bewust niet gekozen voor een ingebouwde rondleiding of *walkthrough*. Op deze manier is het tijdens gebruikerstesten beter te observeren in welke mate de functionaliteit intuïtief ontdekt wordt en waar eventueel extra ondersteuning nodig is.

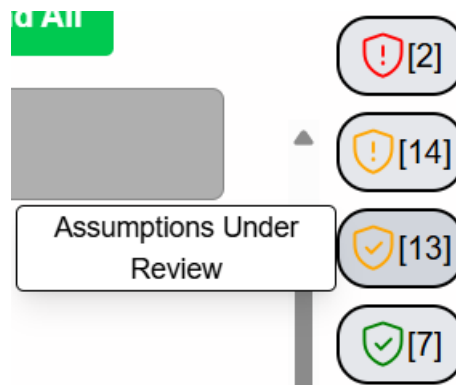


Fig. 16: De hover tooltip bij hoveren over de ingeklapte filter sidebar button

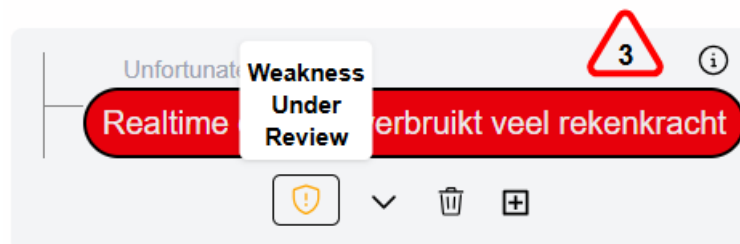


Fig. 17: De hover tooltip bij het hoveren over de iconbutton in de tree lay-out

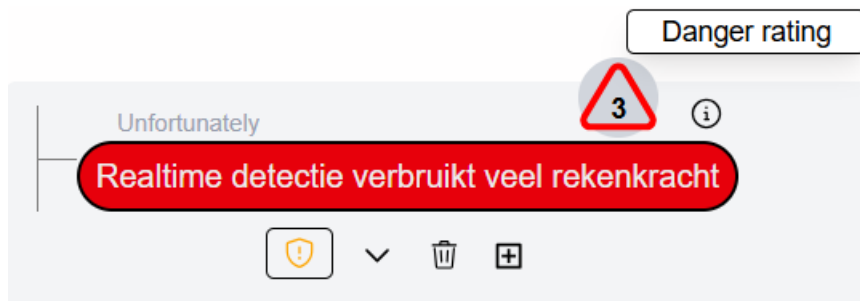


Fig. 18: De hover tooltip bij het hoveren over de danger rating button in de tree lay-out



Fig. 19: De hover tooltip bij het hoveren over de branch summary button in de tree lay-out

3.12 Toepassing van Nielsen's usability-heuristieken in de webtool

Het ontwerp en de implementatie van de fortunately-unfortunately-webtool zijn expliciet afgestemd op de tien usability-heuristieken van Nielsen. De onderliggende React-componenten (Page, TreeVisualizer, SearchBar, IconSelectorButton) realiseren deze heuristieken concreet in de gebruikersinterface.

3.12.1 Zichtbaarheid van de systeemstatus

De tool geeft gebruikers voortdurend feedback over de huidige toestand van het systeem. De boomweergave in TreeVisualizer wordt onmiddellijk geüpdatet na toevoeging, bewerking of verwijdering van nodes via `setTreeData`. De geselecteerde knoop (`currentNode`) en zoekresultaten (`highlightedNodes`) worden visueel geaccentueerd met een extra rand en achtergrondkleur. De shield-knoppen met teller in de rechterzijbalk (`[redNodes.length]`, `[greenNodes.length]` enz.) tonen onmiddellijk na gebruikersinteractie hoeveel zwaktes en aannames zich in elke status bevinden. De "Danger rating"-indicator per unfortunately-knoop (component `NodeDangerPopup`)

geeft continu de actuele risicoscore weer, die direct wijzigt bij aanpassing van impact en likelihood.

3.13.2 Overeenkomst tussen systeem en de echte wereld

De webtool gebruikt terminologie en structuren die aansluiten bij gangbare praktijken in security- en risicoanalyse. Nodes zijn expliciet getypeerd als level "fortunately" of "unfortunately", wat overeenkomt met positieve en negatieve implicaties in het analyseproces. De rechterzijbalk groepeerde nodes in domeinspecifieke categorieën: "Weaknesses", "Weaknesses Under Review", "Assumptions Under Review" en "Verified assumptions", ondersteund door passende schild-iconen (GoShield, GoShieldCheck). De risicoscore wordt opgebouwd uit de dimensies "Impact" en "Likelihood" en weergegeven als "Danger rating", wat aansluit bij gangbare risico-taxonomieën.

3.12.3 Gebruikerscontrole en vrijheid

De implementatie voorziet diverse mechanismen die de gebruiker controle geven over het modelleringsproces. Via "Reset Tree" kan op elk moment naar een lege beginsituatie worden teruggekeerd. Deze actie wordt voorafgegaan door een bevestigingsdialoog (window.confirm) om onbedoeld dataverlies te vermijden. Op knooppniveau kunnen gebruikers afzonderlijke elementen verwijderen via de prullenbakknop (GoTrash) of nieuwe children toevoegen (RiAddBoxLine). Panelen zoals de rechter "shields"-sidebar (isOpen) en het linker "Parents above current node"-overzicht (openParentSummary) kunnen naar wens worden geopend en gesloten, zodat gebruikers vrij kunnen schakelen tussen een compacte en een meer gedetailleerde weergave.

3.12.4 Consistentie en standaarden

In zowel de code als de visuele vormgeving is een consistente logica doorgevoerd. Fortunately-knooppunten worden systematisch in groen weergegeven, unfortunately-knooppunten in rood (component Node en de achtergrondklassen in ShowNodes), een kleurcodering die ook terugkomt in de zijbalk (groene/rode labels met "F" en "U") en in de badges voor gevaar (FiTriangle). De betekenis van iconen blijft

overal gelijk: GoShield markeert zwaktes, GoShieldCheck markeert aannames of voltooide nodes, GoInfo toont branchesamenvatting, en GoChevronDown/Up staan voor uit- en inklappen. Interactie-patronen zoals hover-tooltips, modale bevestigingen en scrollIntoView sluiten aan bij typische webstandaarden, zodat gebruikers bestaande ervaring met andere applicaties kunnen hergebruiken.

3.12.5 Foutpreventie

Het ontwerp tracht fouten te voorkomen door de interactieruimte te beperken tot zinvolle acties. Bij het toevoegen van een child node bepaalt de applicatie automatisch het type (fortunately of unfortunately) op basis van het level van de parent, waardoor semantisch inconsistente ketens vermeden worden. Destructieve acties zoals het verwijderen van een knoop of het resetten van de volledige boom vereisen expliciete bevestiging via window.confirm. Dropdowns voor impact en likelihood zijn begrensd tot waarden 1–5, wat foutieve invoer uitsluit en tegelijk een uniforme schaal afdwingt. Klikken buiten pop-ups (zoals het danger-ratingvenster of de iconselector) sluit deze vensters automatisch, waardoor de kans op onbedoelde verdere interactie in een oude context afneemt.

3.12.6 Herkenning boven herinnering

De interface is zo ontworpen dat relevante acties en informatie zichtbaar worden aangeboden in plaats van uit het geheugen te moeten worden opgeroepen. Contextuele knoppen rond elke knoop (iconselector, expand/collapse, delete, add child) verschijnen bij hover, zodat gebruikers zien welke handelingen mogelijk zijn zonder menu's of commando's te onthouden. De IconSelectorButton toont bij hover een korte tekstuele uitleg ("Weakness", "Assumption Under Review", "Verified assumption", enz.), waardoor de betekenis van de iconen direct herkenbaar is. De SearchBar laat gebruikers zoeken op knoopnaam. De gevonden nodes worden automatisch gehighlight en centraal in beeld gescrold (scrollIntoView), waardoor de gebruiker de exacte positie in de boomstructuur niet hoeft te onthouden.

3.12.7 Flexibiliteit en efficiëntie van gebruik

De tool ondersteunt zowel exploratief gebruik door beginners als efficiëntere workflows voor ervaren gebruikers. Beginners kunnen via de knoppen in de boom en de zijbalk lineair nodes toevoegen en labels aanpassen. Tegelijk kunnen ervaren gebruikers grote modellen snel manipuleren via globale collapse/expand-functionaliteit (CollapseButtons) en door herhaaldelijk op Enter te drukken in het zoekveld om cyclisch door alle matches te navigeren (highlightIndex). De dynamische lijsten van zwaktes en aannames in de rechterzijbalk fungeren als werkvoorraad: een klik op een item zorgt ervoor dat de relevante tak automatisch wordt uitgekapt en in beeld komt, wat gericht en efficiënt werken bij complexe bomen ondersteunt.

3.12.8 Esthetisch en minimalistisch ontwerp

De interface combineert een eenvoudige lay-out met terughoudende visuele elementen. De centrale boomweergave is dominant en gebruikt een beperkt kleurenpalet (groen/rood) om de belangrijkste semantische verschillen te markeren. Secundaire acties verschijnen pas bij hover, waardoor de standaardweergave relatief schoon blijft. De rechterzijbalk is standaard samengevouwen tot compacte “shield”-knoppen. Het volledige overzichtspaneel wordt alleen getoond wanneer de gebruiker hier expliciet voor kiest. Tooltips zijn kort en beperkt tot contextuele uitleg, zodat er geen overdadige tekst of iconografie ontstaat die de cognitieve belasting zou verhogen.

3.12.10 Helpen bij herkennen, diagnosticeren en herstellen van fouten

Hoewel geavanceerde foutafhandeling nog beperkt is, ondersteunt de tool gebruikers bij het herkennen en corrigeren van fouten. Bevestigingsdialogen bij verwijderen en resetten maken expliciet welke actie op het punt staat uitgevoerd te worden, zodat de gebruiker een vergissing kan terugdraaien door te annuleren.

In de NodeDangerPopup worden de actuele danger rating, impact en likelihood expliciet getoond, waardoor incongruente waarden makkelijk opgemerkt en aangepast kunnen worden. De branch-samenvatting (via de GoInfo-knop en getParentsAbove) helpt gebruikers bij het diagnosticeren van structurele fouten in de redeneringsketen: door de

context van een knoop te tonen, wordt het eenvoudiger vast te stellen of een veronderstelling op de juiste plaats staat.

3.12.11 Help en documentatie

Tot slot voorziet de webtool in lichte, geïntegreerde ondersteuning. Verschillende elementen bevatten inline-tooltips die bij hover verschijnen, zoals “Danger rating” bij de rode driehoek, “Show branch summary” bij het infopictogram en semantische labels bij de icons in IconSelectorButton (“Weakness Under Review”, “Completed node”, “Low Importance”). De zoekbalk is duidelijk gelabeld (“Search for node”) en gebruikt een herkenbare Material-UI TextField, waardoor de functie intuïtief is. Deze vorm van embedded documentatie reduceert de noodzaak voor een omvangrijke externe handleiding en ondersteunt vooral eerste-gebruikssessies.

3.13 Prototypebeperkingen

De huidige codebasis is opgezet als proof-of-concept:

- opslag gebeurt lokaal en er is nog geen integratie met een backend of externe threat-databanken;
- er is geen multi-user synchronisatie, waardoor collaboratieve workflows buiten scope vallen;
- performance-optimalisaties voor extreem grote bomen zijn beperkt tot basismaatregelen (zoals uitklapbare takken).

Deze keuzes houden de implementatie beheersbaar en laten toe om de impact van de visualisatie- en UX-beslissingen in de gebruikersstudie te evalueren, zonder afgeleid te worden door infrastructuur of integraties.

Hoofdstuk 4

Gebruikersstudie

4.1 Doel van de gebruikersstudie

De studie had als doel de bruikbaarheid, duidelijkheid en efficiëntie van de dreigingsmodelleringstool te evalueren aan de hand van twee scenario's, namelijk het uitwerken van een securitymodel voor een inlog- en authenticatiesysteem als scenario één en het analyseren van een AI-fraudedetectiesysteem als scenario twee. Er werden twee versies van de tool ontwikkeld: een basisversie en een versie met extra functionaliteiten. Elke versie was gekoppeld aan één van de scenario's, om zo een vergelijking te kunnen maken tussen beide varianten.

4.2 Verloop van de gebruikersstudie

Deelnemers kregen eerst een introductie tot de fortunately–unfortunately-methode en de bijkomende statussen (Weakness, Weakness Under Review, Assumption Under Review, Verified Assumption) en danger rating. Vervolgens doorliepen ze de twee scenario's. Hierbij voerden de deelnemers gestructureerde scenario-taken uit in de software en vulden nadien een User Experience Questionnaire (UEQ) en open vragen in. De volledige sessie duurde gemiddeld tussen één uur en één uur en een kwartier.

Bij de gebruikers werd afgewisseld in de volgorde van de scenario's: de ene helft (4 deelnemers) werkte eerst scenario 1 uit en daarna scenario 2, terwijl de andere helft (4 deelnemers) de omgekeerde volgorde volgde. De pilotstudie werd uitgevoerd met eerst de basisversie en daarna de versie met extra functionaliteiten. Deze wisseling van volgorde had tot doel de invloed van de learning curve van de fortunately–unfortunately-methode op de resultaten te minimaliseren.

4.3 Deelnemers

In totaal namen acht deelnemers deel aan de hoofdgebruikersstudie, aangevuld met één deelnemer in een voorafgaande pilotstudie. De leeftijd van de deelnemers in de hoofdstudie varieerde van 20 tot 28 jaar en allen hadden een achtergrond in (toegepaste) informatica of industriële wetenschappen informatica.

Zeven van de acht deelnemers in de hoofdstudie gaven expliciet aan géén ervaring te hebben met security-modellering. Eén deelnemer rapporteerde beperkte ervaring met security-modellering in het kader van opleiding en non-functionele security-requirements. Wel hadden de meeste deelnemers enige ervaring met security in bredere zin, bijvoorbeeld via security-vakken, een cybersecurity-challenge of een lopend vak cybersecurity. De huidige functies omvatten studenten, (junior) software developers, een data engineer/consultant en een scrum master/functional analyst, wat overeenkomt met de beoogde doelgroep van ontwikkelaars met basiskennis maar zonder diepgaande security-expertise.

De pilootdeelnemer had een vergelijkbare opleiding (industriële wetenschappen informatica), maar beschikte wél over expliciete ervaring met security-modelleringstechnieken zoals STRIDE en attack trees en werkte als MES-software engineer. De pilot werd gebruikt om het studieprotocol en de taken te verfijnen. De gegevens van de pilootdeelnemer zijn selectief meegenomen in de analyses. Voor de User Experience Questionnaire (UEQ) werd deze deelnemer wél opgenomen, omdat hij/zij op basis van de uitgevoerde taken beide versies van de tool inhoudelijk kon beoordelen en zo extra inzicht gaf in de ervaren gebruikservaring. Voor de prestatiematen, zoals taakduur en aantal gemaakte fouten, is de piloot echter uitgesloten om vertekening door het eerdere testkarakter van de sessie te vermijden.

4.4 Softwareversies

De basisversie van de tool bevat enkel kernfunctionaliteit: nodes toevoegen, nodes verwijderen, alle nodes inklappen, alle nodes uitklappen, per node inklappen en een zoekfunctie. Figuur 21 toont de visualisatie van deze basisversie. In deze versie moeten de status (Weakness, Weakness Under Review, Verified Assumption, Assumption Under Review) en soort van een node (fortunately/unfortunately) expliciet in de node-naam zelf worden opgenomen door de gebruiker. Figuur 22 toont de visualisatie van een fortunately-node in de basisversie, terwijl Figuur 23 de visualisatie van een unfortunately-node weergeeft.

De uitgebreide versie bouwde hierop voort met bijkomende visualisatie- en analysefeatures, zoals kleurcodering voor het soort node, iconen voor classificatie, filtering op status, telling van nodes per status, rangschikking op basis van danger rating, highlighting vanuit de filterbalk, samenvattingsfunctionaliteit en tooltips. Beide versies gebruikten dezelfde onderliggende fileview tree-representatie van het fortunately–unfortunately-model, geïmplementeerd in de codebasis van de prototypeapplicatie.

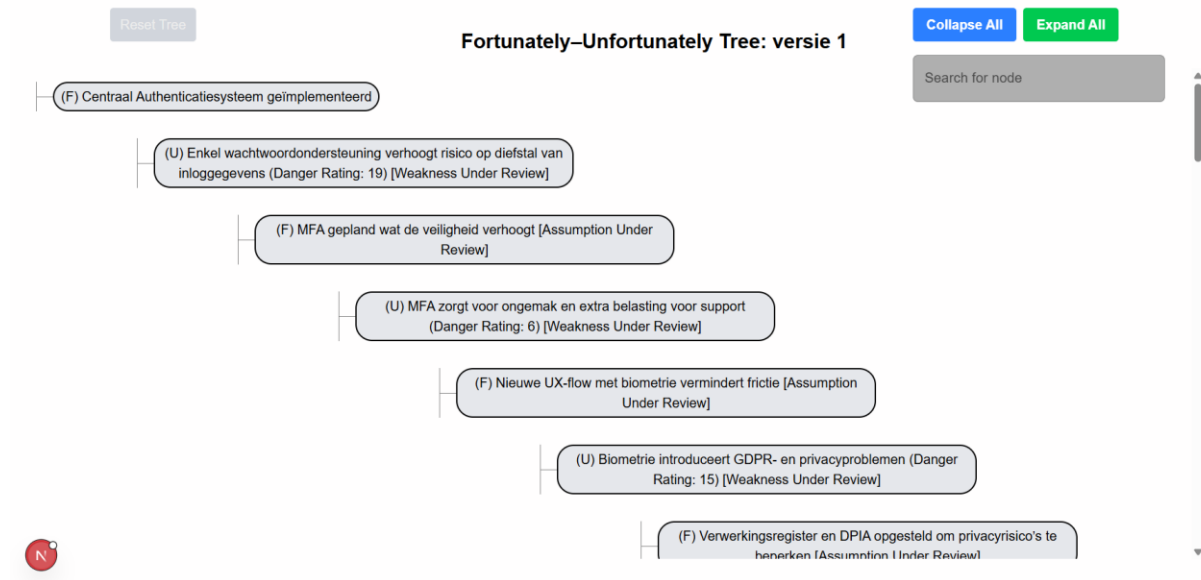


Fig. 20: De visualisatie van de tree structuur en de features in de basisversie van de software

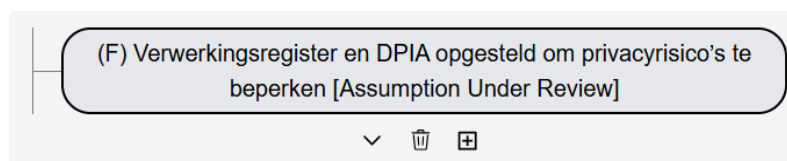


Fig. 21: De fortunately-node weergaven in de basisversie van de software



Fig. 22: De unfortunately-node weergaven in de basisversie van de software

4.5 Taken en procedure

Per scenario kregen deelnemers een gedeeltelijk ingevuld model en moesten zij dit verder verkennen, analyseren en aanvullen via vier opeenvolgende taken. Typische subtaken waren: ontbrekende unfortunately-nodes toevoegen, nieuwe fortunately-tegenmaatregelen uitwerken, danger ratings invullen, statussen correct toekennen en lijstjes van specifieke nodecategorieën opstellen (bijvoorbeeld alle weakness-nodes of de weakness under review met hoogste danger rating).

Daarnaast voerden deelnemers taken uit om informatie uit de tree te halen, zoals het opzoeken van alle Verified Assumption-nodes of het reconstrueren van de keten van nodes van de root tot een specifieke node. In de laatste taak van elke story moest de deelnemer een samenvatting opstellen van de volledige fortunately–unfortunately-keten tot aan een gekozen of gegeven node, als maatstaf voor de ondersteuning van de tool bij communicatie en documentatie.

Een voorbeeld van de taken voor een scenario zijn:

1. Informatie opzoeken: Lijst de volgende informatie op (stop na eerste 3 woorden per node): alle Weakness-nodes, Verified Assumption-nodes, Weakness Under Review met hoogste danger rating, aantal Assumption Under Review-nodes;
2. Model uitbreiden en kwantificeren: Voeg onder 'Uitrol van wachtwoordmanager...' de Unfortunately node 'Sommige werknemers weigeren wachtwoordmanagers' (Low Importance) toe, markeer bovenliggende Fortunately als Verified Assumption. Onder 'Provider-storingen...' voeg de fortunately node 'Team richt on-prem fallback...' toe, de unfortunately node 'On-prem fallback synchroniseert...' (likelihood 4, impact 3, danger rating 12) en de fortunately node 'Periodieke synchronisatie...' toe;
3. Keten reconstrueren: Zoek 'Junior teamleden besluiten alle servers uit te schakelen' en documenteer de volledige keten van root tot deze node voor teamdiscussie;
4. Alternatieve paden exploreren: Onder 'Hoge false positives blokkeren legitieme gebruikers' voeg de fortunately node 'Model afgestemd om gemiste fraude...' (Assumption Under Review) en de Unfortunately node 'Het terugdringen van false negatives...' (likelihood 5, impact 1, danger rating 5) toe.

4.6 Dataverzameling

Tijdens de sessie werd het scherm van de deelnemer opgenomen en live gevolgd, zodat interactiepatronen en fouten later kwalitatief konden worden geanalyseerd. Na elke versie vulden deelnemers de User Experience Questionnaire (UEQ)[<https://www.ueq-online.org/>] in en beantwoordden ze open vragen over welke features nuttig waren, welke verbeterd konden worden en welke versie ze in een reële werksituatie zouden verkiezen.

De studie verzamelde zo zowel kwantitatieve gebruikservaringsscores als kwalitatieve feedback over de toegevoegde visualisatiefeatures ten opzichte van de basisfunctionaliteit.

4.7 User Experience Questionnaire

De UEQ is een gevalideerde, gestandaardiseerde vragenlijst op basis van semantische tegenstellingen, ontworpen voor een integrale evaluatie van gebruikerservaring. De vragenlijst omvat zes schalen: aantrekkelijkheid (algemene waardering), duidelijkheid (gemak van leren en begrijpen), efficiëntie (taakuitvoering zonder overmatige inspanning), betrouwbaarheid (gevoel van controle en voorspelbaarheid), stimulatie (mate van motivatie en plezier) en nieuwheid (creativiteit en interesse). De schalen worden beoordeeld via gepaarde tegenstellingen op een 7-puntsschaal, waarbij hogere scores wijzen op een positievere gebruikerservaring.

De User Experience Questionnaire (UEQ) biedt een gevalideerde, kwantitatieve methode om gebruikerservaringen van threat modeling-tools te evalueren, waarbij pragmatische (duidelijkheid, efficiëntie, betrouwbaarheid) en beleavingsgerichte schalen (stimulatie, nieuwheid) worden gecombineerd. De tool ondersteunt eenvoudige statistische analyses en is breed toegepast in uiteenlopende HCI-contexten.

4.8 Open vragen en rangschikking

In dit onderdeel werden een reeks open vragen en een rangschikkingsopdracht gebruikt om dieper inzicht te verkrijgen in de ervaren meerwaarde van de nieuwe functionaliteiten ten opzichte van de basissoftware. De vragen richtten zich op zowel de

begrijpelijkheid van de gehanteerde methode (de “Fortunately–Unfortunately”-aanpak) als op de perceptie van nut en bruikbaarheid van de toegevoegde features.

De deelnemers werd eerst gevraagd of de gebruikte methode duidelijk was, om zo de basis van de evaluatie te controleren. Vervolgens moesten zij de nieuwe features rangschikken van meest nuttig (1) tot minst nuttig (8). Wanneer een feature niet was opgemerkt, konden zij dit aanduiden met een score van –1. Hierdoor kon een onderscheid gemaakt worden tussen functies die effectief gebruikt werden en functies die minder opvielen tijdens de takenuitvoering.

De daaropvolgende open vragen waren bedoeld om kwalitatieve feedback te verzamelen over mogelijke verbeterpunten in de uitgebreide versie van de software, alsook suggesties voor aanvullende of alternatieve functies die de gebruikerservaring kunnen versterken.

Tot slot werd gevraagd in welke versie de deelnemers de taken als makkelijker uitvoerbaar ervoeren en welke versie zij zouden kiezen in een reële werksituatie. Deze informatie biedt inzicht in de algemene voorkeur en de perceptie van gebruiksgemak tussen de softwareversies.

Hoofdstuk 5

Resultaten

In dit hoofdstuk worden de resultaten van de gebruikersstudie besproken. De analyse richt zich op de vergelijking tussen de basisversie en de versie met extra functionaliteiten van de dreigingsmodelleringsstool. De bevindingen wijzen erop dat versie 2, met de toegevoegde features, over het algemeen als gebruiksvriendelijker, efficiënter en aantrekkelijker werd ervaren dan versie 1.

5.1 Kwantitatieve UX-resultaten

In de UEQ-resultaten (Fig. 24) scoort versie 2 op alle dimensies (aantrekkelijkheid, transparantie, efficiëntie, bestuurbaarheid, stimulatie en originaliteit) duidelijk positief, terwijl versie 1 op meerdere schalen rond neutraal of uitgesproken negatief blijft. Vooral stimulatie en originaliteit maken een grote sprong van sterke negatieve beoordelingen in versie 1 naar duidelijke positieve beoordelingen in versie 2. Dit wijst op een veel plezierigere en moderner aanvoelende interactie. Ook efficiëntie en aantrekkelijkheid verschuiven van negatieve naar positieve scores. Dit suggereert dat gebruikers versie 2 als sneller, minder foutgevoelig en visueel aantrekkelijker ervaren.

In de UEQ-resultaten wordt dit bevestigd door de gemiddelden: versie 1 laat negatieve scores zien voor aantrekkelijkheid (-0,83), efficiëntie (-0,72), stimulatie (-0,97) en originaliteit (-0,78), terwijl transparantie (0,39) en bestuurbaarheid (0,53) slechts licht positief zijn. In versie 2 verschuiven deze scores naar duidelijke positieve waarden: aantrekkelijkheid 1,04; transparantie 1,72; efficiëntie 1,33; bestuurbaarheid 1,25; stimulatie 0,69 en originaliteit 1,25. De resultaten wijzen op een algemeen aanzienlijk positievere gebruikerservaring.

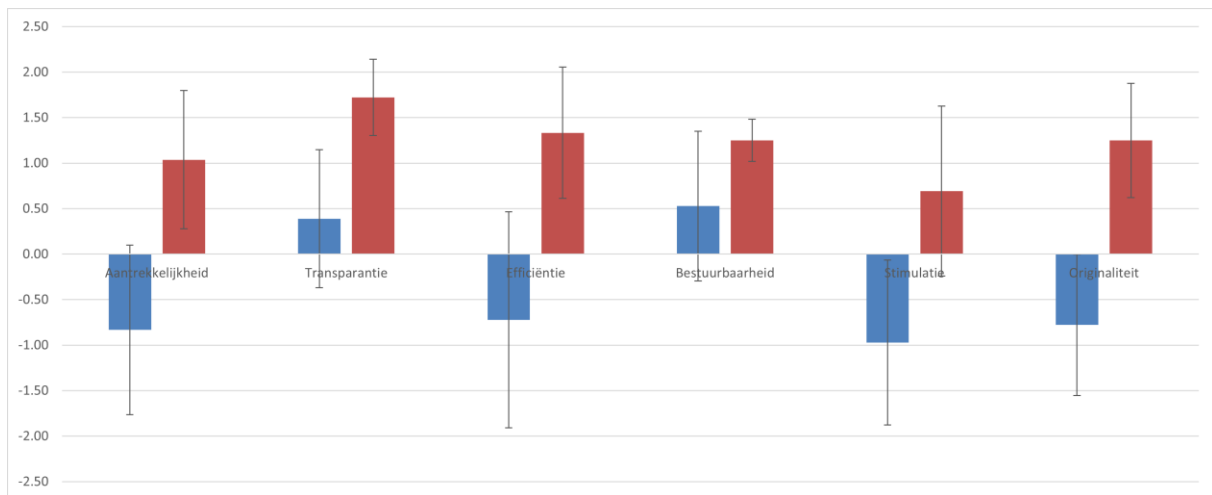


Fig. 23: Weergave van de UEQ resultaten in een grafiek met de gemiddelde waarden en hun t-test waarden

5.2 Kwalitatieve observaties

Uit de schermopnames bleek dat deelnemers in de basisversie (versie 1) veel tijd verloren met scrollen en visueel zoeken naar relevante nodes. De zoekbalk werd vaak gebruikt om specifieke termen terug te vinden, maar typefouten zorgden er geregeld voor dat resultaten uitbleven. Hierdoor moesten de deelnemers langer zoeken en hadden ze soms meerdere pogingen nodig om de juiste node(s) te lokaliseren.

In de basisversie moesten de deelnemers het aantal nodes per status manueel bepalen door herhaald gebruik van de zoekbalk. Daarnaast vergaten deelnemers regelmatig om na het toevoegen van nodes meteen een status of danger rating toe te kennen. Dit zorgde ervoor dat zij moesten terugkeren naar eerder aangemaakte nodes om deze informatie alsnog in te vullen. Dit bleek zowel tijdrovend als foutgevoelig te zijn. In de uitgebreide versie (versie 2) kwam dit probleem minder vaak voor. De krijgen standaard al een status mee en de danger rating is visueel prominenter aanwezig. Dit zorgde ervoor dat het overslaan van deze stap sneller opviel.

Ook bij het reconstrueren van branches viel een duidelijk verschil op. In de basisversie moesten deelnemers de volledige keten manueel in de boom volgen om een samenvatting te noteren. Hierdoor werd het proces trager en foutgevoeliger. Met de branch summary in versie 2 konden zij dezelfde informatie rechtstreeks in een lineaire weergave aflezen. Het reconstrueren van ketens vergt minder inspanning en verliep merkbaar vlotter.

5.3 Waardering van de toegevoegde features

De antwoorden op de open vragen sluiten nauw aan bij deze kwantitatieve resultaten. Deelnemers geven unaniem aan dat zij de taken makkelijker konden uitvoeren met versie 2 en noemen daarbij expliciet de aanwezigheid van filter- en sorteerfunctionaliteit, automatische danger-ratingberekening en zoekmogelijkheden. Versie 2 vermindert de cognitieve belasting doordat gebruikers niet langer zelf hoeven te tellen of danger ratings manueel moeten berekenen. Meerdere deelnemers beschrijven dat de nieuwe functies tijd besparen en de kans op fouten verkleinen. De filtering op basis van node-status komt herhaaldelijk als meest nuttig naar voren, gevolgd door het tellen van nodes per status en de highlight-functie bij klikken in de filterbalk.

5.4 Ervaren gebruiksgemak en voorkeur

Ook op visueel en belevingsmatig vlak wordt versie 2 duidelijk hoger gewaardeerd. Deelnemers omschrijven deze versie als visueel aantrekkelijker, met overzichtelijke kleuren, een duidelijkere GUI en iconen die de gebruiker door de taakflow begeleiden. Dit komt overeen met de verhoogde UEQ-scores op aantrekkelijkheid, stimulatie en originaliteit en bevestigt dat de nieuwe vormgeving niet alleen mooier oogt, maar ook de oriëntatie binnen de boomstructuur ondersteunt. Alle deelnemers geven aan dat zij in een realistische werksituatie versie 2 zouden verkiezen, omdat deze volgens hen efficiënter, duidelijker, makkelijker in gebruik en functioneler is dan versie 1.

5.5 Verbeterpunten en suggesties

Ondanks de duidelijke voorkeur voor versie 2 formuleren deelnemers ook concrete verbeterpunten, wat waardevolle input biedt voor verdere iteraties van het ontwerp. Genoemde suggesties zijn onder meer het compacter weergeven van de tree zodat er meer op het scherm past, intuïtievare manieren om nodes in en uit te klappen en snellere navigatie binnen lange branches. Daarnaast worden uitbreidingen rond statusbeheer voorgesteld, zoals custom statussen, een duidelijkere indicatie van de collapse/expand-knoppen en een eenvoudiger manier om per ongeluk aangemaakte

nodes te annuleren. Tot slot worden ideeën geopperd voor verdere kwaliteitsverbeteringen, zoals een globale knop om alle branches behalve de geselecteerde in te klappen, zoom op een specifieke branch als nieuwe tree en een zoekbalk binnen de filterbalk.

5.6 Synthese

Samenvattend tonen zowel de UEQ-scores als de open antwoorden dat het toevoegen van de nieuwe features leidt tot een verschuiving van een overwegend negatief naar een duidelijk positief UX-profiel. Versie 2 wordt door alle deelnemers als voorkeursversie beschouwd omdat deze tijd bespaart, fouten reduceert en een aantrekkelijker, overzichtelijker en intuïtiever ontwerp biedt, terwijl de geformuleerde suggesties tegelijk concrete aangrijpingspunten bieden voor verdere verfijning van de interface.

5.7 Samenvatting van de belangrijkste resultaten

De empirische resultaten bevestigen dat de basisfunctionaliteit van de tool voldoende is voor minimale ondersteuning van Fortunately-Unfortunately trees, maar dat de toegevoegde UX-features een substantiële verbetering opleveren in gebruiksvriendelijkheid en efficiëntie. Versie 2 scoort op alle UEQ-dimensies positief en wordt door alle deelnemers unaniem verkozen boven de basis mindmap-tool, dankzij features zoals filtering en visuele codering. Deze bevindingen beantwoorden de onderzoeksvragen direct en onderbouwen de waarde van domeinspecifieke UX in threat modeling.

5.8 Beantwoording van de onderzoeksvragen

5.8.1 Onderzoeksvraag 1: Minimale basisfunctionaliteit

De basisversie (versie 1) ondersteunt essentiële taken zoals node-creatie, uitklappen/inklappen en zoeken, wat noodzakelijk is voor het opbouwen van Fortunately-Unfortunately trees. Echter, de negatieve UEQ-scores op efficiëntie,

stimulatie en originaliteit tonen aan dat deze functionaliteit alleen ontoereikend is voor soepele threat modeling zonder security-expertise. Dit sluit aan bij de literatuur over drempels in threat modeling, waar generieke tools cognitieve belasting verhogen door gebrek aan domeinspecifieke ondersteuning.

5.8.2 Onderzoeksvraag 2: Meest nuttige extra functies

Filtering op node-status scoort consequent als meest nuttig (meerdere rang 1), gevolgd door node-tellingen per status en highlighting vanuit de filterbalk. Visuele hulpmiddelen zoals kleurcodering en iconen (schilden) worden hoog gewaardeerd voor overzicht, terwijl tooltips en branch-samenvattingen lager scoren maar nog steeds positief bijdragen aan navigatie. Deze voorkeur bevestigt dat pragmatische features (zoeken, sorteren) voorop staan bij complexe hiërarchische data, in lijn met Nielsen's heuristieken voor flexibiliteit en herkenning.

5.8.3 Onderzoeksvraag 3: Vergelijking met een basisversie

Versie 2 overtreft de basisversie van de mindmap-software op gebruiksvriendelijkheid (positieve UEQ-scores vs. negatief voor basisversie) en effectiviteit (tijdsbesparing, foutreductie via automatisering). Deelnemers benadrukken dat domeinspecifieke elementen zoals automatische danger ratings en status-filters threat modeling toegankelijker maken voor niet-experts, wat de kloof tussen theorie en praktijk adresseert.

Hoofdstuk 6

Discussie

6.1 Relatie met literatuur en theorie

De resultaten van dit onderzoek sluiten aan bij en nuanceren bevindingen uit recente literatuur over security-visualisatie en hiërarchische boomrepresentaties. Devendra et al. tonen in hun systematische review dat de huidige praktijk gedomineerd wordt door graph-based en notation-based visualisaties (netwerkgrafen, dependencygrafen, UML/ER-diagrammen, enz.), voornamelijk gericht op run-time en infrastructuur-gerichte security-analyse zoals log-inspectie, netwerkmonitoring en user-behaviour analytics. Tegelijk signaleren zij dat veel van deze tools moeilijk te integreren zijn in agile/CI-omgevingen en weinig ondersteuning bieden voor niet-experts, waardoor de cognitieve belasting hoog blijft. De in deze thesis ontwikkelde Fortunately–Unfortunately-tool positioneert zich expliciet in dit gat: in plaats van runtime-data te tonen, ondersteunt de visualisatie het denkproces tijdens threat modeling, met een fileview-achtige boomstructuur, danger ratings en filtering die volgens de UEQ-resultaten de ervaren efficiëntie en stimulatie duidelijk verhogen ten opzichte van een generieke mindmap-baseline.[7]

Vanuit de treemap-literatuur benadrukt Scheibel et al. dat er een rijke design space bestaat van containment-gebaseerde layouts voor hiërarchische data, maar dat de meeste algoritmes optimaliseren voor ruimtelijke efficiëntie, aspect ratio en stabiliteit, niet voor het expliciet tonen van argumentatieve of beslisstructuren. De in deze thesis gebruikte boom sluit formeler aan bij dit containment-principe (elke stap is genest onder zijn ouder), maar gebruikt oppervlakte en positie primair om een sequentiële narratieve keten van “fortunately”- en “unfortunately”-gevolgen leesbaar te maken in plaats van kwantitatieve waarden te encoderen. Daarmee illustreert dit werk hoe inzichten uit treemap-onderzoek kunnen worden hergebruikt in een domeinspecifieke, niet-kwantitatieve boomvisualisatie voor security-ontwerpargumentatie.[5]

Ten slotte sluit de tool inhoudelijk aan bij argument-visualisatiewerk zoals de hi-tree van Sbarski et al., die met bipartite nodes en kleurcodering expliciet steunende en tegenargumenten zichtbaar maken. Net als bij hi-trees staat in de Fortunately–Unfortunately-boom het expliciet visualiseren van inferentiële relaties centraal, maar

dan toegespitst op risico's en mitigaties in security-scenario's, met extra semantiek zoals statusiconen en danger ratings die gericht zijn op ontwikkelaars zonder diepgaande securityachtergrond. De positieve gebruikersfeedback op functies als filtering en branch-samenvattingen suggereert dat dergelijke domeinspecifieke UX-keuzes een concreet antwoord bieden op de door Devendra et al. geschetste nood aan meer user-centric security-visualisaties voor een breder publiek.[6][7]

6.2 Beperkingen van het onderzoek

Kleine steekproef en populatie. De steekproef is klein (9 deelnemers), waardoor de statistische kracht en externe validiteit beperkt zijn en conclusies enkel voorzichtig kunnen worden veralgemeend naar gelijkaardige profielen (ontwikkelaars met informatica-ervaring). Dit risico is deels beperkt door bewust deelnemers te rekruteren met een achtergrond die aansluit bij de beoogde doelgroep van de tool en door taken te formuleren die realistische acties en situaties in een threat-modelingcontext simuleren. Tegelijk blijven belangrijke gebruikersgroepen onderbelicht, zoals security-experts met diepere domeinkennis en absolute leken, zodat vervolgonderzoek met een grotere en meer diverse steekproef nodig is om de bevindingen breder te valideren.

Zelfrapportage en bias. Het gebruik van UEQ-scores en open vragen introduceert subjectiviteit en mogelijke confirmation bias: deelnemers kunnen antwoorden geven die ze als “gewenst” inschatten of achteraf hun ervaring vertekenen. Dit is deels opgevangen door de vragenlijst na concrete taken af te nemen, door observatie van gedrag tijdens de sessies én door de volgorde van de geteste versies per deelnemer te variëren, zodat volgorde-effecten (zoals gewenning of vermoeidheid) beperkt worden. De resultaten moeten daarom vooral geïnterpreteerd worden als indicaties van ervaren bruikbaarheid en voorkeur, niet als harde maat voor objectieve prestatie.

Beperkte vergelijkingsbasis en duur. De studie vergelijkt de prototype-tool enkel met een interne basisversie en niet met gevestigde externe tools zoals MindMeister of Lucidchart, waardoor onduidelijk blijft hoe het ontwerp zich houdt in een breder tool-ecosysteem. Het gebruik van de basisversie bood een gecontroleerd referentiepunt met gelijkaardige functionaliteit en domeinlogica, zonder verschillen in accountvereisten of bijkomende features in externe tools die de resultaten zouden kunnen vertekenen. Bovendien focuste het onderzoek op korte, eenmalige taken, zodat effecten van leercurve, langdurig gebruik en samenwerking niet zijn meegenomen. De bevindingen zijn dus vooral geldig als eerste indicatie van bruikbaarheid binnen deze

specifieke context en niet als evaluatie van langetermijnproductiviteit of concurrentievergelijking.

6.3 Implicaties voor theorie en praktijk

Theoretisch sluit dit werk aan bij design-science-onderzoek, omdat een concreet artefact werd ontworpen, geïmplementeerd en geëvalueerd rond de vraag hoe UX-principes threat modeling kunnen ondersteunen. De resultaten suggereren dat elementen zoals visuele hiërarchie, expliciete statussen en herkenning boven herinnering niet alleen de “gebruiksvriendelijkheid” verhogen, maar ook de toegankelijkheid van security-redeneringen voor ontwikkelaars met beperkte security-expertise. Daarnaast tonen de UEQ-scores en kwalitatieve feedback dat generieke UX-instrumenten zinvol kunnen worden ingezet in de evaluatie van security tools, en dat dimensies zoals duidelijkheid en stimulatie mogelijk een rol spelen in de adoptie binnen DevSecOps-settings.

In de praktijk kan de ontwikkelde tool dienen als laagdrempelige ondersteuning voor DevSecOps-teams die threat modeling in hun bestaande workflows willen integreren zonder zware methodologische drempel. De branch-summary, statusiconen en filterlogica bieden ontwikkelaars een manier om risico’s systematisch te prioriteren, openstaande aannames bij te houden en beslissingen traceerbaar te documenteren, wat aansluit bij iteratieve ontwikkelcycli. Bestaande tools en pipelines kunnen bovendien inspiratie halen uit specifieke ontwerpkeuzes, zoals het expliciet maken van verifieerbare aannames en het combineren van boom-weergave met lijst-gebaseerde prioritering, zonder noodzakelijk het volledige prototype over te nemen.

Voor onderwijs illustreert het prototype hoe Fortunately–Unfortunately-structuren concreet kunnen worden ingezet om studenten redeneringsketens rond risico’s, mitigaties en afhankelijkheden te laten verkennen. Docenten kunnen de tool gebruiken als oefenomgeving waarin studenten bestaande modellen analyseren of zelf een threat-model opbouwen, met directe visuele feedback via iconen en danger ratings. Dit kan helpen om abstracte concepten uit threat modeling tastbaarder te maken en de kloof tussen theoretische methodes en praktische toepassing in softwareprojecten te verkleinen.

6.4 Aanbevelingen voor toekomstig onderzoek

Een eerste aanbeveling is om vervolgonderzoek uit te voeren met grotere en meer diverse steekproeven, inclusief security-experts en gebruikers zonder technische achtergrond, om de generaliseerbaarheid van de bevindingen te vergroten.

Longitudinale studies waarin teams de tool gedurende langere tijd in reële projecten gebruiken, zouden bovendien inzicht kunnen geven in leercurves, duurzame adoptie en effecten op samenwerking en documentatiekwaliteit.

Ten tweede is het zinvol om de tool systematisch te vergelijken met commerciële mindmap- en diagramsoftware via gecontroleerde A/B-studies. Door het prototype naast tools zoals MindMeister of Lucidchart te plaatsen, kan beter worden bepaald in welke mate de specifieke UX-features (zoals branch-summary, statussen en danger ratings) daadwerkelijk meerwaarde bieden boven generieke mindmapping-functionaliteit, zowel qua efficiëntie als qua begrip van bedreigingen en mitigaties.

Verder onderzoek kan zich ook richten op de uitbreiding van functionaliteit die in deze studie slechts gedeeltelijk of conceptueel werd uitgewerkt. Features zoals collaps-children-knoppen, branch-zoom en real-time multi-user-samenwerking zijn vooral relevant voor grotere modellen en distribute teams, maar vragen afzonderlijke evaluatie op vlak van bruikbaarheid en performance. Ten slotte vormen AI-gebaseerde aanbevelingen voor threats en mitigaties een interessant spoor: automatische suggesties kunnen de drempel voor minder ervaren gebruikers verlagen, maar roepen tegelijk vragen op rond betrouwbaarheid, transparantie en mogelijke over-afhankelijkheid, wat zorgvuldig empirisch onderzoek vereist.

Hoofdstuk 7

Conclusie

Threat modeling is een systematische aanpak om mogelijke bedreigingen, kwetsbaarheden en misbruikscenario's in een softwaresysteem te identificeren en te mitigeren vóór implementatie. In de praktijk blijft deze stap echter vaak achterwege door de perceptie dat threat modeling complex, tijdsintensief en voorbehouden aan security-experts is. De Fortunately–Unfortunately-methode biedt een meer verhalende, iteratieve manier om risico's en mitigaties als ketens van “gelukkig/helaas”-stappen te structureren, maar er ontbrak een toegankelijke, gespecialiseerde tool om deze aanpak in DevSecOps-context te ondersteunen. In dit werk werd daarom een domeinspecifieke webtool ontworpen, geïmplementeerd en geëvalueerd, met als doel threat modeling laagdrempeliger te maken voor ontwikkelaars met beperkte security-expertise.

Centraal stonden de volgende drie onderzoeksvragen:

1. welke basisfunctionaliteit minimaal noodzakelijk is om threat modeling met Fortunately–Unfortunately-trees te ondersteunen;
2. welke bijkomende UX-features verhogen de bruikbaarheid en verlagen de cognitieve belasting;
3. hoe verhoudt een uitgebreide versie van de tool zich tot een basisversie op vlak van gebruiksvriendelijkheid en effectiviteit.

Om deze vragen te beantwoorden werd eerst een basisprototype ontwikkeld met kernfunctionaliteit (nodecreatie, inklappen/uitklappen, zoeken), gevolgd door een uitgebreide versie met onder meer statusiconen, danger ratings, branch-summary en filterzijpanelen. Beide versies werden vervolgens vergeleken in een gebruikersstudie met ontwikkelaars, waarbij zowel kwantitatieve metingen (UEQ-scores, taakprestaties) als kwalitatieve feedback werden verzameld.

Met betrekking tot onderzoeksvraag 1 tonen de resultaten dat de minimale basisfunctionaliteit weliswaar voldoende is om Fortunately–Unfortunately-trees structureel op te bouwen, maar ontoereikend blijft voor efficiënte threat modeling zonder bijkomende ondersteuning. De negatieve UEQ-scores van versie 1 en de gerapporteerde moeite om overzicht te houden op grotere trees suggereren dat gebruikers snel het noorden verliezen en veel tijd kwijt zijn aan navigatie en handmatig prioriteren. Dit bevestigt dat een “platte” mindmap-benadering niet volstaat om niet-experts effectief door complexere security-redeneringen te loodsen.

Voor onderzoeksvraag 2 blijkt uit zowel de vragenlijsten als de observaties dat gebruikers vooral de pragmatische UX-features waarderen: filtering op node-status, tellingen per status en directe highlighting vanuit de filterbalk worden als bijzonder nuttig ervaren, gevolgd door visuele codering via kleuren en iconen. Deze elementen helpen gebruikers om snel de belangrijkste weaknesses en aannames te identificeren, de voortgang van hun model in te schatten en relevante delen van de boom terug te vinden zonder langdurig scrollen. De resultaten wijzen erop dat dergelijke features de cognitieve belasting merkbaar verlagen door “herkenning boven herinnering” te ondersteunen en onnodige zoek- en navigatieacties te beperken.

Bij onderzoeksvraag 3 laat de vergelijking tussen de basisversie en de uitgebreide versie zien dat de doorgevoerde UX-verbeteringen zich vertalen in zowel subjectieve als objectieve winst. De uitgebreide tool (versie 2) scoort duidelijk positiever op de meeste UEQ-schalen dan de basisversie, en deelnemers rapporteren een hogere mate van controle, overzicht en efficiëntie. Tegelijkertijd wijzen observaties en taakmetingen op tijdswinst en minder fouten bij het lokaliseren en beoordelen van risico's, wat erop wijst dat threat modeling met deze uitgebreide tool toegankelijker wordt voor niet-experts dan met een generieke mindmap-opzet.

Overkoepelend voldoet de tool in belangrijke mate aan bekende usability-heuristieken van Nielsen, zoals zichtbaarheid van systeemstatus, overeenstemming tussen systeem en mentale modellen, en minimalisatie van geheugenlast. Het UX-profiel verschuift van overwegend negatief in de basisversie naar duidelijk positief in de uitgebreide versie, met name op dimensies als efficiëntie, stimulatie en originaliteit. Praktisch impliceert dit dat development teams vergelijkbare UX-patronen – filtering op status, automatische danger ratings, expliciete statusiconen en branch-samenvattingen – kunnen integreren in bestaande threat-modelingtools of DevSecOps-pipelines om de drempel tot structurele security-analyses te verlagen.

Voor de praktijk biedt het werk daarmee een concreet, laagdrempelig startpunt voor organisaties die threat modeling willen verankeren in hun ontwikkelprocessen zonder zware methodologische of toolmatige instap. Voor het onderwijs vormt het prototype een didactische omgeving waarin studenten Fortunately–Unfortunately-ketens kunnen opbouwen en analyseren, en zo op een visuele en interactieve manier kennismaken met kernconcepten uit threat modeling. Toekomstig onderzoek kan voortbouwen op deze basis door de tool te testen in grotere en meer diverse settings, de voorgestelde verbeteringen (zoals globale collapse-knoppen, branch-zoom en real-time samenwerking) te implementeren en de rol van AI-ondersteuning voor threat-suggesties verder te verkennen. Zo kan de hier gepresenteerde tool uitgroeien van onderzoeksprototype tot een volwaardig instrument voor zowel industrie als security-onderwijs.

Referentielijst

- [1] A. Shostack, “Fast, Cheap and Good An Unusual Trade-off Available in Threat Modeling,” 2021. Accessed: Jan. 11, 2026. [Online]. Available: <https://shostack.org/files/papers/Fast-Cheap-and-Good.pdf>
- [2] N. Shevchenko, T. A. Chick, P. O’riordan, T. P. Scanlon, and C. Woody, “Threat Modeling: A Summary of Available Methods,” *Dtic.mil*, Jul. 2018. <https://apps.dtic.mil/sti/html/tr/AD1084024/>
- [3] Z. Shi, K. Graffi, D. Starobinski, and N. Matyunin, “Threat Modeling Tools: A Taxonomy,” *IEEE Security & Privacy*, vol. 20, no. 4, pp. 2–13, 2021, doi: <https://doi.org/10.1109/msec.2021.3125229>.
- [4] Hendrik Ewerlin, “😄 Fortunately $\bowtie$ 😞 Unfortunately of 😄 Fortunately $\bowtie$ 😞 Unfortunately,” *Threat-modeling.net*, 2026. <https://threat-modeling.net/fortunately-unfortunately-of-fortunately-unfortunately/>
- [5] W. Scheibel, D. Limberger, and J. Döllner, “Survey of treemap layout algorithms,” in *the 13th International Symposium on Visual Information Communication and Interaction*, New York: Association for Computing Machinery, Aug. 2020. Accessed: Dec. 08, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/3430036.3430041>
- [6] P. Sbarski, T. van Gelder, K. Marriott, D. Prager, and A. Bulka, “Visualizing Argument Structure,” *Lecture Notes in Computer Science*, vol. 5358, pp. 129–138, 2008, doi: https://doi.org/10.1007/978-3-540-89639-5_13.
- [7] I. Devendra, C. Wijesiriwardana, and P. Wimalaratne, “State-of-the-Art in Software Security Visualization: A Systematic Review,” *arXiv (Cornell University)*, Sep. 2025, doi: <https://doi.org/10.48550/arxiv.2509.20385>.